



DEFINIÇÃO DO FRAMEWORK E MoT

Para o WissTek-IoT

Fase 3 da TpM

Sumário

1	Considerações Iniciais	4
2	Modelo de Referência para IoT.....	5
2.1	Modelo de Referência para IoT CISCO	5
2.2	Modelo de Referência para IoT Open Source (OSRM-IoT)	6
3	TpM	7
4	Framework e TpM	8
5	Framework TpM.....	11
6	Tempo Real e Tempo não Real.....	12
6.1	Exibição de gráficos	15
6.2	Definição dos Níveis.....	16
6.3	Diagrama Temporal de fluxo de dados	17
7	Captura e Coleta de Dados Brutos – Tempo Real.....	20
8	Topologias de RSSF.....	21
9	MAC Distribuída e Centralizada	23
9.1	Comunicação Bidirecional Half Duplex	23
9.2	MAC Distribuída.....	24
9.3	MAC Centralizada	25
10	Operações Nível 3	26
11	Pilha de Protocolo MoT.....	29
11.1	Pilha de Protocolo	29
11.2	Camada de aplicação – APP	30
11.3	Camada de Transporte - TRANSP	30
11.4	Camada de Rede - NET	30
11.5	Camada de acesso ao meio – MAC.....	31
11.6	Camada física – PHY	31

12	Framework com Pilha de Protocolo	32
12.1	Camada de Aplicação e Níveis do Framework	32
12.2	Independência da camada física com MoT	33
13	Implementação com Kit-LoRa	35
13.1	Fluxo de dados N3 e N1	35
13.2	Definição Nó Sensor e Gateway	Erro! Indicador não definido.
13.3	Definição do Nó sensor – N1 e N2.....	36
13.4	Definição do Gateway N2 Camada Física	37
14	Pilha MoT com LoRa.....	39
15	GitHub – Framework Basic	40
16	Códigos Nó Sensor e Gateway	41
16.1	MoT no Nó Sensor	41
16.2	Gateway.....	42
17	Pythons e Dados armazenados.....	44
17.1	N2_N3.py – MoT e Aplicação.....	46
17.2	N4 - Armazenamento.....	46
17.2.1	Comando do LED Amarelo.....	47
17.2.2	Dados Brutos.....	47
17.2.3	Dados de Visualização	48
17.3	N5 - Processamento.....	49
17.4	N6 – Exibição.....	51
18	Exemplo Passo a Passo do Framework Básico	51
18.1	Projeto do Nó sensor	52
18.2	Pacote MoT DL e UL.....	53
18.3	Pacote de DL	54

18.4	Pacote de UL.....	55
18.5	Acionamento do LED Amarelo	57
18.6	Python N2 e N3	57
18.7	Leitura do Comando do LED Amarelo	58
19	Organização Dados Brutos.....	67

1 Considerações Iniciais

O Framework é parte integrante da TpM para a implementação de uma solução IoT (S-IoT) e a definição de um protocolo de comunicação chamado Management over Tunnelling (MoT).

Esse documento estabelece as definições do **Framework** da TpM e do protocolo MoT.

Esse será o referencial para ser utilizado por todos os trabalhos no WissTek-IoT.

Não é objetivo aqui entrar em detalhe sobre a TpM, apenas contextualizar seu uso e, mas mostrar como o Framework é utilizado pela TpM.

Apenas será feito um resgate conceitual do que é um modelo de referência para IoT e da TpM e como se chegou a um Framework de implementação.

Para definições mais práticas foi criado um **Framework Básico** com um exemplo de utilização para medida de luminosidade e acionamento de um LED amarelo.

O objetivo é o de apresentar um exemplo completo de implementação de solução IoT, que é a Fase 3 da TpM.

Esse é um exemplo simples para consolidar conceitos e serão a base para os trabalhos do WissTek-IoT.

São incluídos todos os componentes necessários em cada nível do modelo de referência e sua interação.

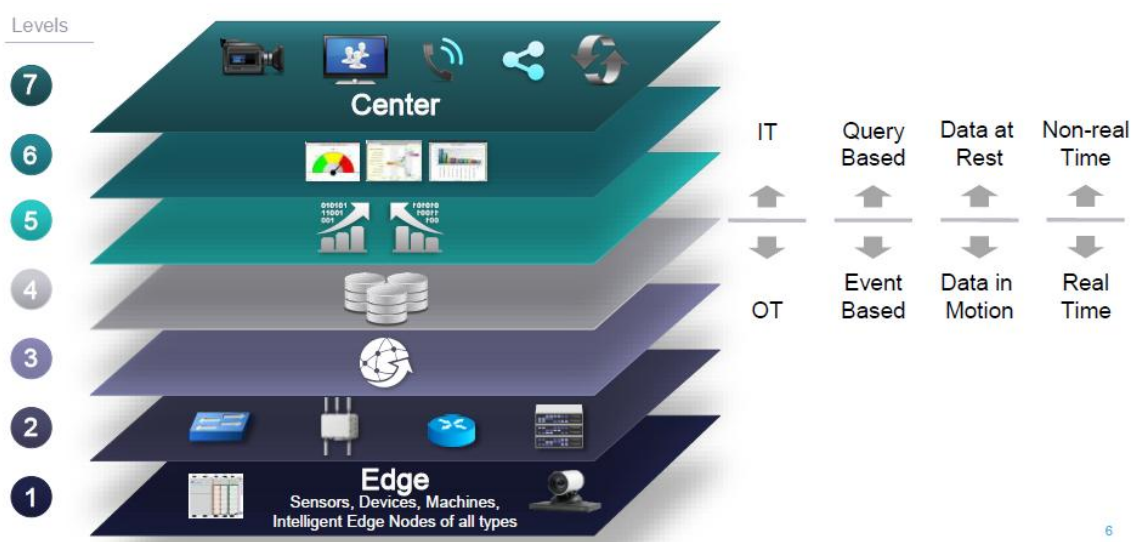
2 Modelo de Referência para IoT

Para entender o Framework Básico é necessário primeiramente entender o que significa Modelo de Referência para IoT.

A base conceitual da S-IoT é entender que existe um modelo de referência para IoT utilizado pela TpM, que foi inspirado no Modelo de Referência para IoT da CISCO e criado um open source.

2.1 Modelo de Referência para IoT CISCO

Essa definição foi inspirada na CISCO, sendo muito apropriada para o Framework Básico, como mostrado na próxima figura.



Um ponto fundamental é separar as partes que operam em tempo real, chamada de OT (Object Technology), e as partes que operam não em tempo real IT (Information Technology).

Observar que essa divisão se dá no Nível 4 – armazenamento.

A divisão entre tempo real e tempo não real é feita no Nível 4 – Armazenamento, que identifica as 4 funções de armazenamento:

- Tempo real
 - Dados brutos são aqueles coletados em tempo real pelo Nível 3 e armazenados por ele no Nível 4.

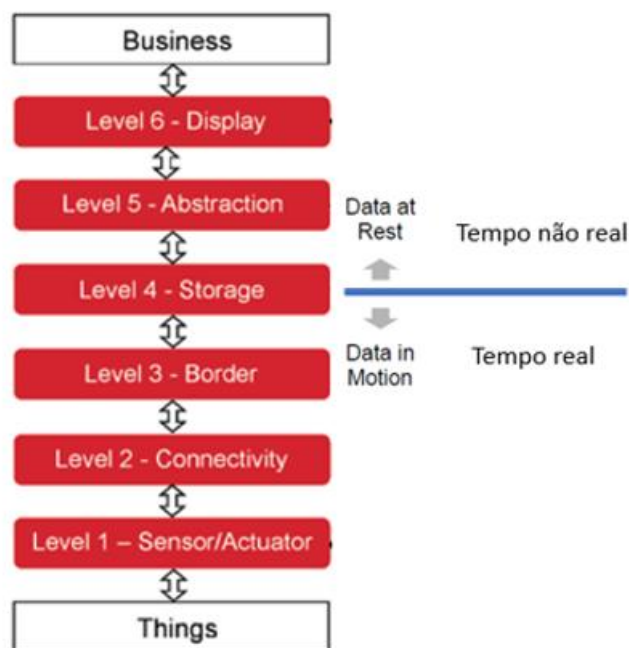
- Blockchain é o armazenamento em tempo real através de blocos.
- Tempo não real
 - Dados processados pelo Nível 5.
 - Parâmetros para execução o Nível 1.

Para o estabelecimento da comunicação entre o Nível 3 e o Nível 1, através do Nível 2, foi criado o MoT (Management over Tunnelling).

Portanto, a atividade em tempo real depende de um protocolo de comunicação que é o MoT.

2.2 Modelo de Referência para IoT Open Source (OSRM-IoT)

Foi criado um modelo de referência, que é utilizado pelo WissTek-IoT com seis níveis.



Atenção na divisão entre processos que estão em tempo real e processo que estão em tempo não real que está no Nível 4.

3 TpM

Antes de entrar na parte prática é necessário entender em que contexto o Framework se enquadra.

Para criar a solução IoT foi criada no laboratório WissTek-IoT da Unicamp a TpM como definido no artigo do Carlinho.



Research article

The Three-Phase Methodology for IoT Project Development[☆]

Luiz Carlos B.C. Ferreira^{a,*}, Pedro R. Chaves^b, Raphael M. Assumpção^b, Omar C. Branquinho^c, Fabiano Fruett^b, Paulo Cardieri^a

^a Communications Department, Campinas State University, Campinas, Brazil

^b Semiconductors Instruments and Photonics Department, Campinas State University, Campinas, Brazil

^c PoB-Tec and the Extension Program of Computing Institute, Campinas State University, Campinas, Brazil

A Figura 1 desse artigo apresenta como a TpM é utilizada como metodologia para criar uma solução IoT.

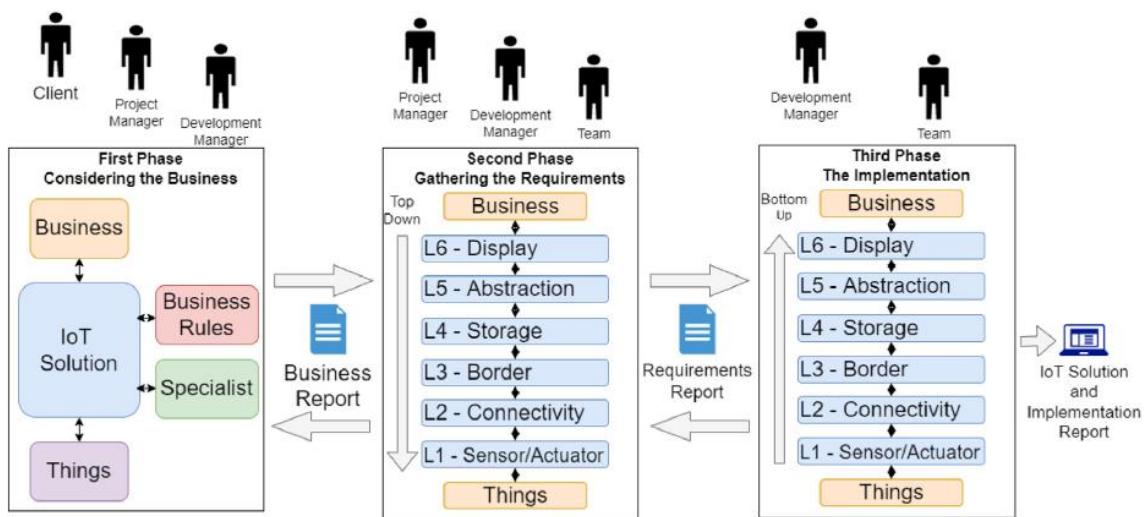


Fig. 1. TpM-Pro iterations diagram showing roles, phases and deliverables.

4 Framework e TpM

A TpM originalmente não estabelecia como seria a Fase 3 da TpM

O artigo mostrado no item anterior apresenta a TpM como uma metodologia para orientar a criação de uma solução IoT.

Porém, não foi especificado como implementar efetivamente a solução IoT.

Para atender essa demanda foi criado o **Framework da TpM** com uma visão horizontal do modelo de referência para IoT.

O Framework foi proposto no artigo do Carlinho do SIoT de 2025.

A Systemic View on Implementing IoT Solutions

Luiz Ferreira
IFSULDEMINAS - Poços de Caldas
Poços de Caldas, Brazil
<https://orcid.org/0000-0003-4102-9742>

Pedro Chaves
Bosch Group
Campinas, Brazil
<https://orcid.org/0000-0003-4279-8850>

Raphael Assumpção
FEEC - Unicamp
Campinas, Brazil
<https://orcid.org/0000-0001-8438-642X>

Omar Branquinho
FEEC - Unicamp
Campinas, Brazil
<https://orcid.org/0000-0002-5193-8643>

Paulo Cardieri
FEEC - Unicamp
Campinas, Brazil
<https://orcid.org/0000-0002-7761-0240>

O Framework é uma visão sistêmica de uma solução IoT fazendo parte da Fase 3 da TpM.

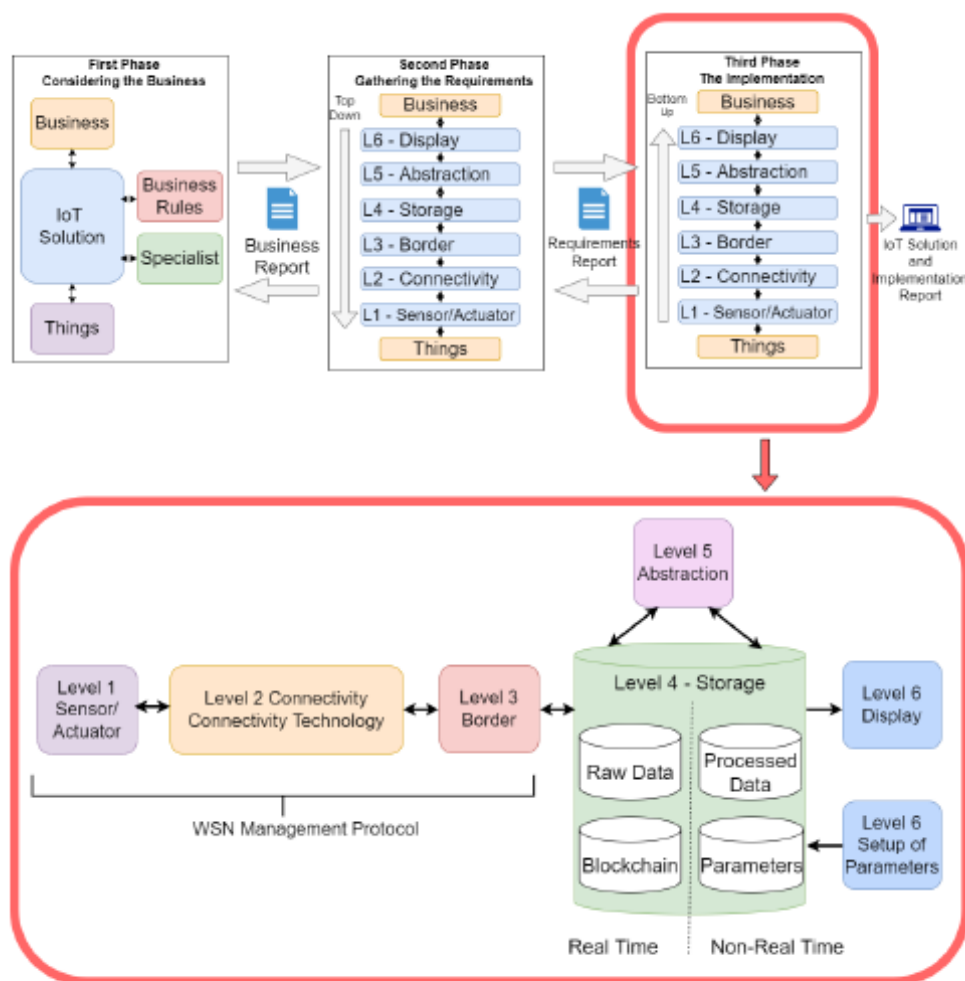


Fig. 4. TpM Framework

Como mostrado a figura que está na Figura 4 do artigo.

Para contextualizar a posição do Framework na TpM é necessário entender que ele somente é utilizado após as fases da TpM:

- Fase 1 – Negócio: análise do negócio, regras de negócio, áreas especialistas e definição das coisas.
- Fase 2 – Requisitos: utiliza o modelo de referência: exibição, abstração, armazenamento, borda, conectividade e coisas.
- Fase 3 – Implementação
 - 3.1 – Definição das tecnologias que utiliza a mesma visão vertical de baixo para cima: coisas, conectividade, borda, armazenamento, abstração e exibição

- 3.2 – Implementação através do Framework que apresenta uma visão horizontal das partes do modelo de referência: processos locais, conectividade, borda, armazenamento, abstração e exibição.

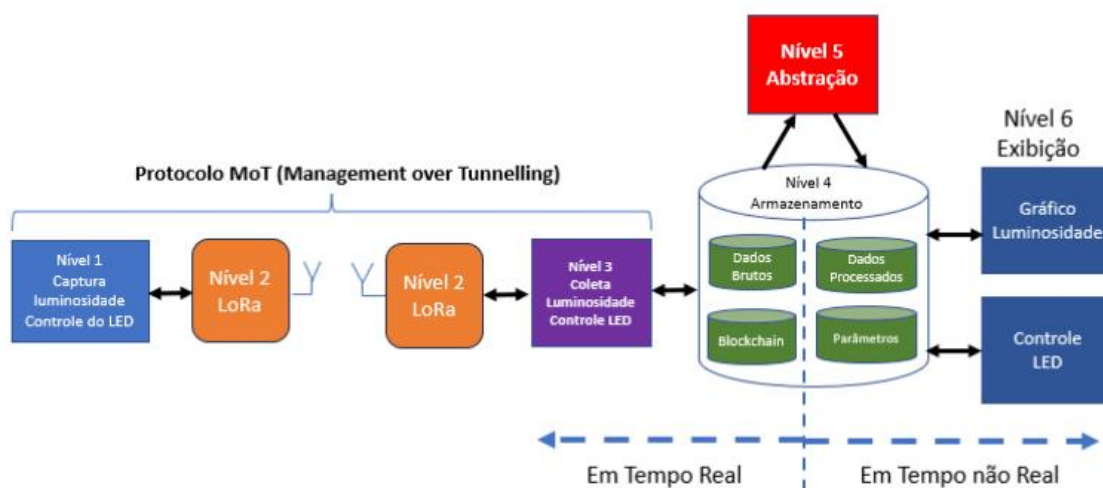
Essa visão sistêmica tem por finalidade apresentar as partes da solução IoT indicando como será implementada.

5 Framework TpM

A implementação da solução IoT deve estabelecer como as tecnologias serão utilizadas para criar a solução, estabelecendo as questões de projeto de cada uma das partes.

O **Framework**, portanto, não identifica a tecnologia, mostrando apenas as funções que devem ser atendidas por cada um dos níveis.

Na próxima figura o Framework utiliza um código de cores para cada um dos níveis do modelo de referência, facilitando as explicações de cada nível.



Esse código de cores será especialmente importante para as definições necessárias para o Nível 2 – conectividade.

Analisando cada nível:

Nível 1 – faz a medida da luminosidade e comanda o LED amarelo.

Nível 2 – tecnologia de comunicação para transmitir e receber, que será o LoRa.

Nível 3 – coleta a luminosidade vinda do Nível 1 e envia comando para acionar LED no Nível 1.

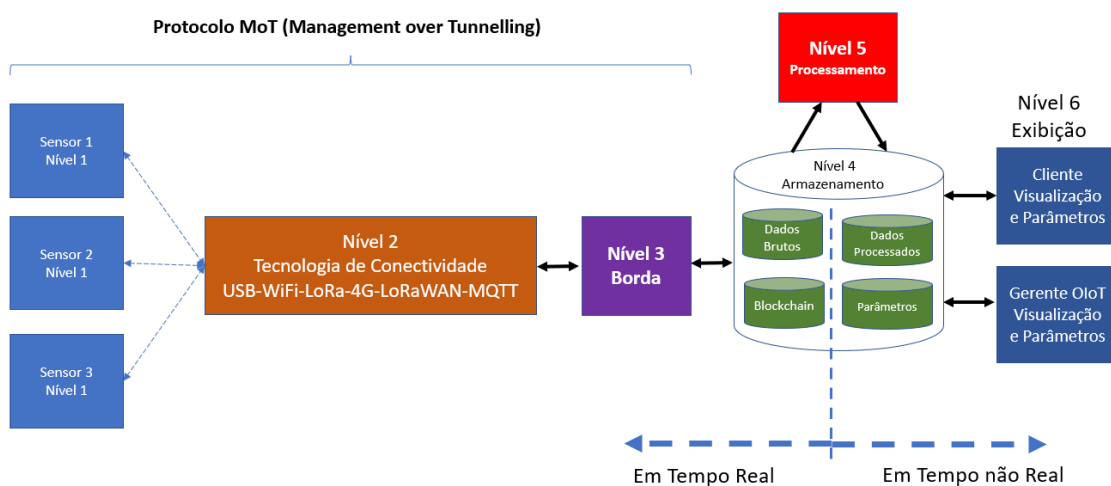
Nível 4 – armazenamento de dados em tempo real e em tempo não real.

Nível 5 – processamento dos dados brutos para gerar os dados processados.

Nível 6 – exibição de gráfico da luminosidade e entrada de parâmetro para acionar o LED amarelo.

Essa divisão lógica das funções de cada nível é muito importante, pois permite identificar as tecnologias.

Embora nesse material seja mostrado apenas um nó sensor o Framework permite vários sensores.



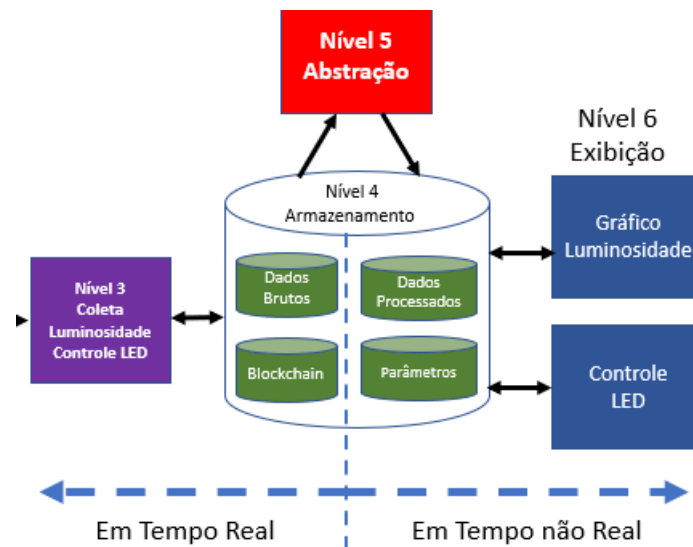
6 Tempo Real e Tempo não Real

Toda S-IoT funciona com uma parte em tempo real e outra parte em tempo não real.

O Nível 4 faz o armazenamento os seguintes tipos de dados:

- Dados brutos são os dados coletados pelo Nível 3 e blockchain, esse último não será tratado nesse documento.
- Dados processados são gerados pelo Nível 5, manipulando os dados brutos, e armazenando os dados processados no Nível 4.

- Parâmetros são comandos para serem executados no Nível 1, lidos e enviados pelo Nível 3.

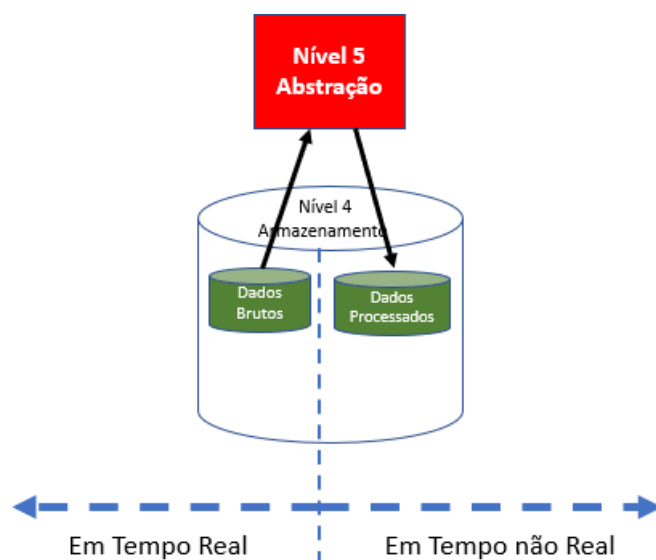


Esse é um ponto **fundamental** de entender que não existe a ligação direta entre os níveis 3, 5 e 6, sendo essa comunicação feita através de dados armazenados no Nível 4.

Os níveis 3, 5 e 6 são independentes e funcionam de forma descorrelacionadas.

O Nível 5 é responsável pelo processamento dos dados brutos armazenados pelo Nível 3 e o resultado é armazenado no Nível 4 como dados processados.

Basicamente o Nível 5 faz a leitura dos dados brutos (pacotes DL e UL) e processa esses dados e armazenando o resultado como dados processados.

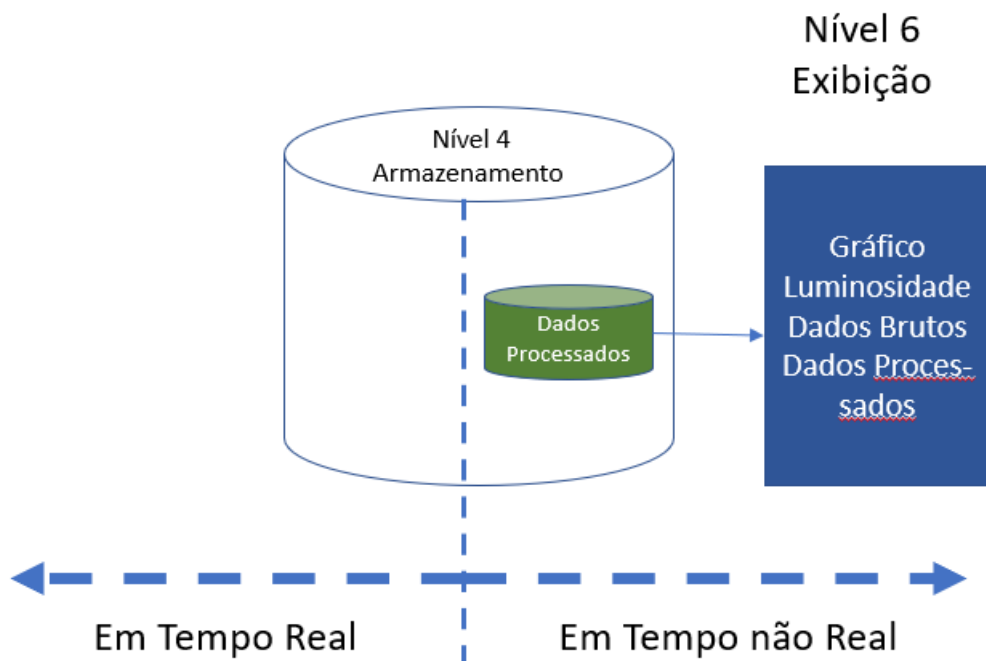


O Nível 6 – Exibição – é dividido em duas partes:

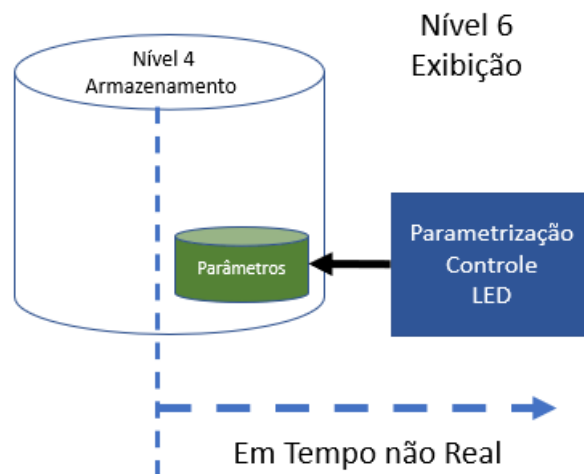
- Exibição de gráficos com dados em tempo real e dados processados.
- Exibição de entrada de dados de controle e outras informações que vão alimentar as outras partes do Framework.

6.1 Exibição de gráficos

Todos os gráficos ou resultados que são exibidos estão nos **Dados Processados**.



Armazenamento dos comandos para serem executados no Nível 1. No exemplo é comando do LED amarelo.



Outros tipos de parâmetros podem ser:

- Limiares para decisão, por exemplo para acionamento do LED amarelo.
- Condições de algoritmos de processamento do Nível 5.

6.2 Definição dos Níveis

No exemplo apresentado na figura anterior existe uma sequência de ações que são desempenhadas pelos níveis do modelo de referência.

O N6 é a interface com o cliente e é dividido em duas partes:

- Exibição de gráfico da luminosidade e da média da luminosidade.
- Tela para opção de ligar o desligar o LED amarelo.

O N5 faz o processamento dos dados brutos e não será considerado nesse primeiro exemplo.

Basicamente o N5 processa os dados brutos armazenados pelo N3, que são exibidos no N6.

Por exemplo, a média da luminosidade que pode ser calculada a cada novo valor armazenado.

O N4 é tem uma função lógica de armazenamento e estabelece a interface entre os níveis 3, 5 e 6.

No exemplo serão usados 3 tipos de armazenamentos:

- Arquivo TXT com o status do LED, em função da opção do cliente.
- Arquivo TXT com os dados brutos da luminosidade coletada pelo N3. A cada nova medida é incluída no arquivo TXT.
- Arquivo TXT com a média da luminosidade, a cada novo valor da luminosidade é calculada a média de todos os valores anteriores e incluído no arquivo.

O Nível 3 é implementado em conjunto com a pilha de protocolo 2 através de um Python que implementa o N2 e N3 é o que faz a dinâmica da solução IoT.

O Nível 2 é a tecnologia de conectividade, que no exemplo é a interface LoRa

O Nível 1 trata dos processos locais, no exemplo o acionamento do LED amarelo e a medida da luminosidade.

6.3 Diagrama Temporal de fluxo de dados

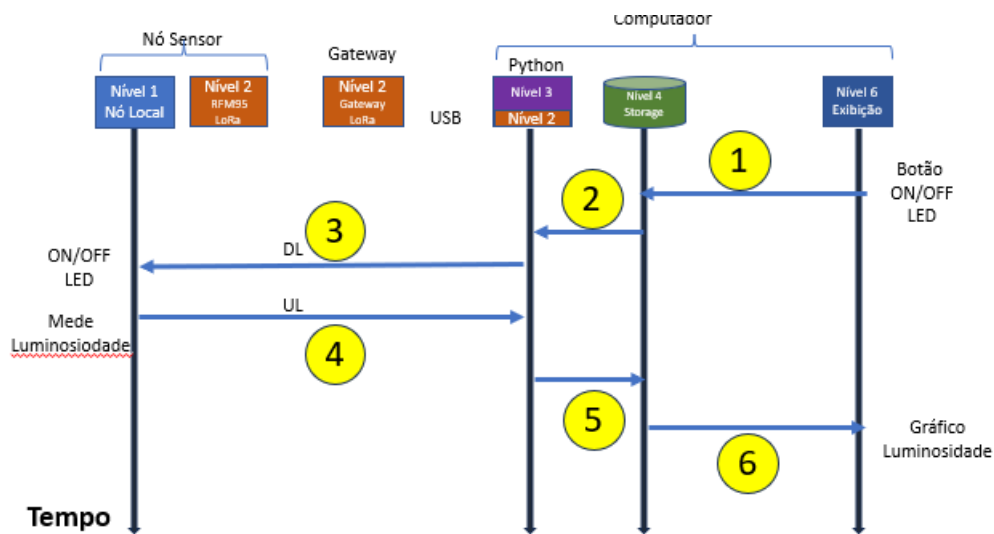
Para entender o fluxo dos dados a seguir está o diagrama temporal do fluxo dos dados.

Observar que os processos são assíncronos, o que significa dizer que acontecem de forma independente.

A seguir é mostrada a sequência de um acesso centralizado em que o N3 inicia a comunicação:

- DL: N6-N4-N3-N2-N1
- UL: N1-N2-N3-N4-N6

O Nível 5 será tratado a parte.



Fluxo dos dados assíncronos.

1 – Cliente tem um botão, implementado com Python, que pode ligar ou desligar o LED.

A opção do cliente é escrita em um arquivo no Nível 4 com valor 0 para LED apagado e 1 para LED aceso.

Nome do arquivo `cmd_led_amarelo.txt`

Ao clicar no botão salvar cria um arquivo de uma linha com valor 0 ou 1.

2 – Python implementa o Nível 3 e Nível 2. Isso é chamado de Borda lê o arquivo `cmd_led_amarelo.txt` e coloca no pacote byte 34 o valor 0 para desligar LED ou 1 para ligar o LED.

3 – Camada física do Python do Nível 2 envia pela USB para ESP32 que por sua vez envia para o módulo RFM95 via SPI.

4 – Nó sensor recebe o pacote DL pelo RFM95 que via SPI passa para camada física do EXP32.

5 – Pacote sobe a pilha MoT do nó sensor e na camada de aplicação e aciona LED se byte 16 for 1 ou apaga se for zero.

6 - Captura a luminosidade na camada de aplicação e coloca nos bytes 18 e 19 (inteiro e resto) e desce a pilha de protocolo e transmite o pacote de UL para o Gateway.

7 – Python Nível 3 recebe a luminosidade vinda o gateway Nível 2 e armazena em um arquivo chamado luminosidade.txt que vai registrando em tempo real as luminosidades.

8 – Python Nível 6 exibe luminosidade e RSSI de DL e UL.

Define-se essa uma estratégia de acesso centralizada utilizado pelo exemplo.

7 Captura e Coleta de Dados Brutos – Tempo Real

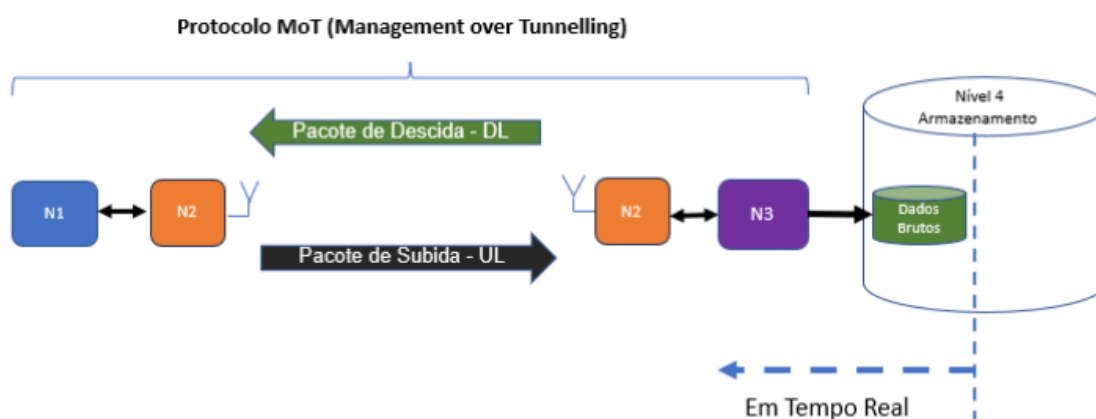
Esse material tem foco na RSSF que vai ser criada com a tecnologia LoRa, que interliga o Nível 3 com o Nível 2.

O Nível 3 é o coração do Framework e cria toda a dinâmica de funcionamento, coletando dados de medidas e enviando comandos.

Dados brutos são os dados coletados pelo Nível 3 vindos do Nível 1, através de uma tecnologia de conectividade do Nível 2, no exemplo o LoRa.

São dados brutos sem nenhum processamento e são armazenados no nível 4 em tempo real.

A próxima figura detalha a obtenção dos dados brutos.



Na figura é identificado o protocolo MoT (Management over Tunnelling) que estabelece a forma como os Níveis 3 e 1 se comunicam, criando uma disciplina de comunicação da RSSF.

Os dados brutos armazenados no Nível 4 serão os próprios pacotes DL e UL que serão processados pelo Nível 5.

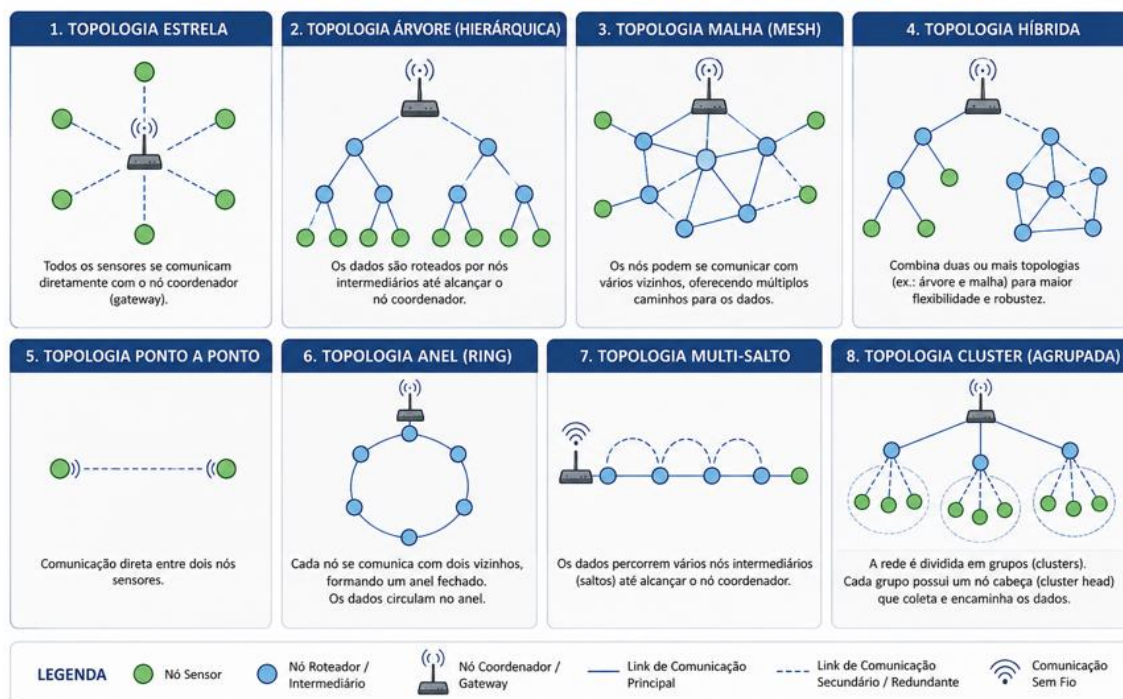
A organização dos nós sensores e o tipo de RSSF é chamado de topologia da rede, que podem ser várias, como mostrado a seguir.

8 Topologias de RSSF

O Nível 2 com a tecnologia de comunicação cria uma rede de sensores sem fio (RSSF).

A organização da RSSF é o que se denomina como topologia, que podem ser de diferentes formas, como mostra a figura.

TIPOS DE TOPOLOGIAS DE REDES DE SENSORES SEM FIO



É preciso estabelecer a disciplina para a transferência das informações dos nós sensores para o gateway.

Basicamente é identificar qual elemento, gateway ou nó sensor, inicia a comunicação.

Para o Framework é definida a disciplina de comunicação como um controle de acesso ao meio MAC (Medium Access Control), que pode ser distribuído ou centralizado.

Nesse material será utilizada a topologia estrela, com um gateway que se comunica com algum tipo de disciplina com os nós sensores.

•



É preciso estabelecer qual a disciplina para a transferência das informações dos nós sensores para o gateway.

9 MAC Distribuída e Centralizada

MAC significa o controle de acesso ao meio e nesse material é definidas suas funções.

Nesse material será utilizada a topologia estrela.

A MAC tem as informações relacionadas a tempo entre pacotes e energia dos nós sensores, por exemplo.

São definidas as técnicas de acesso MAC com duas possibilidades:

- MAC Distribuída
- MAC Centralizada

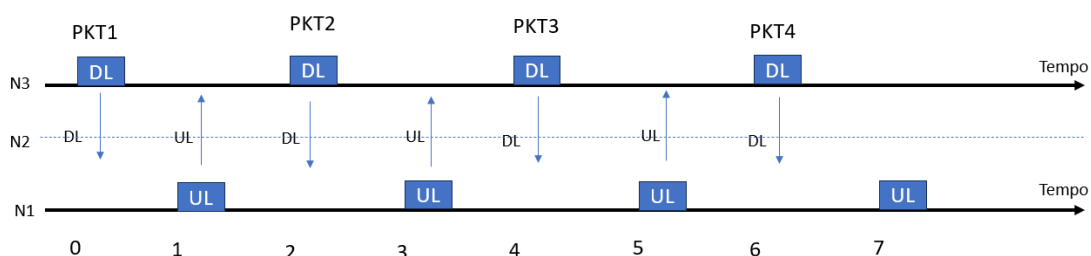
Para o Framework é definida a disciplina de comunicação como um controle de acesso ao meio MAC (Medium Access Control), que pode ser distribuído ou centralizado.

9.1 Comunicação Bidirecional Half Duplex

A comunicação na faixa de 915 MHz, em que opera o LoRa, é Half-duplex, ou seja, cada pacote ocupa tempos diferentes.

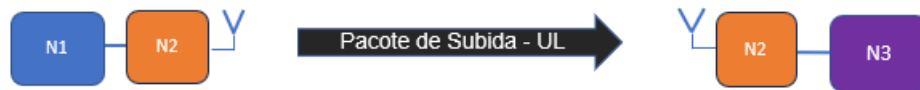
Portanto, não existe acesso ao meio simultâneo de gateway e nó sensores.

A próxima figura mostra que não existem pacotes transmitidos ao mesmo tempo.



9.2 MAC Distribuída

Na **MAC distribuída** a comunicação se inicia com o Nó sensor, ou seja, o Nível 1 gera um pacote e transmite para o Nível 3, denominado pacote de uplink UL, ou pacote de subida.



O problema dessa técnica distribuída é colisão de pacotes quando existe um número grande de sensores, com a necessidade de estratégias de mitigação de colisão.

Outra questão é quanto à estratégia de controle quando o Nível 3 deve enviar um comando para o Nível 1, por exemplo acionar o LED, necessitando uma coordenação entre os níveis.

Um exemplo dessa técnica é o nó sensor Classe A do LoRaWAN, que muitas vezes é vendido somente com a capacidade de UL.

Nesse exemplo de solução não seria possível acionar o LED amarelo, por não ter o pacote de DL.

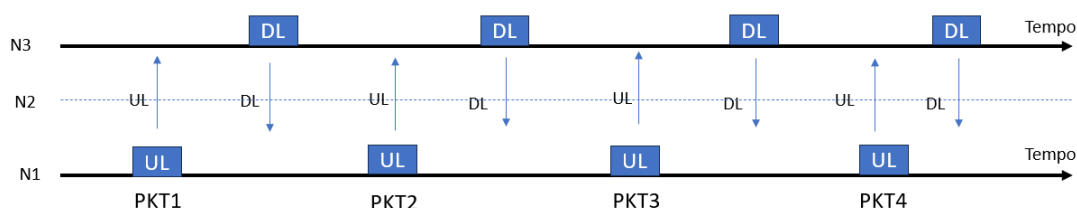
Existe a opção de DL no Classe A do LoRaWAN, entretanto é necessária uma temporização para acessar o nó sensor Classe A que abre duas janelas para recebimento de dados.

Definimos no WissTek-IoT que se não existir um pacote de DL não se configura classicamente como uma rede de sensores sem fio que necessariamente deve ser bidirecional, permitindo o envio de controle para o Nível 1.

Em resposta ao pacote enviado pelo Nível 1 o Nível 3 pode ou não enviar um pacote de DL.



No WissTek essa não é a solução preferencial pois existe uma descoordenação da comunicação dos nós sensores causando colisão.

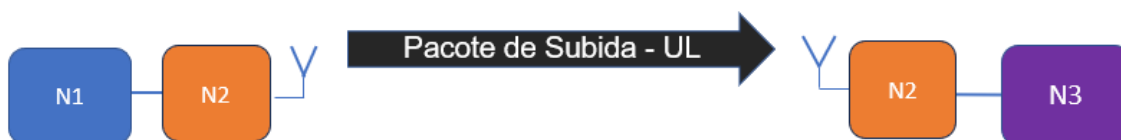


9.3 MAC Centralizada

Na **MAC centralizada** a iniciativa de comunicação é pelo gateway, ou seja, o Nível 3 gera um pacote de comando do LED amarelo, chamado pacote de downlink (DL) e envia para o Nível 1.

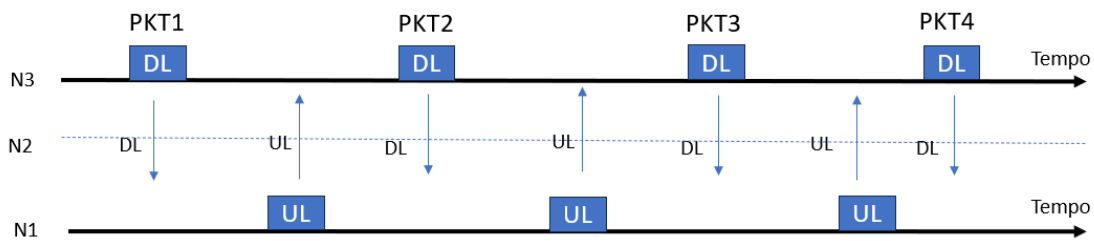


O Nível 1 responde com um pacote de uplink (UL) com a informação solicitada, no caso a luminosidade.



Nesse tipo de MAC não existem as duas questões mencionadas anteriormente, colisão e envio de comandos de controle, uma vez que o Nível 3 que envia um pacote e o Nível 1 responde.

Será utilizada a MAC centralizada para a implementação da solução IoT de demonstração nesse material.



10 Operações Nível 3

O Nível 3 terá 2 funções:

- Coletar os dados oriundos dos nós sensores, no exemplo coletar a luminosidade, e armazenar como dados brutos no Nível 4.
- Ler no Nível 4 qual comando deve ser enviado para o nó sensor, no exemplo acionar o LED amarelo.

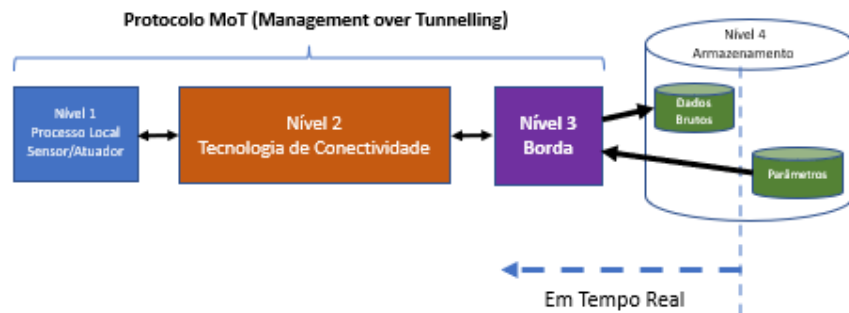
O protocolo MoT é responsável por organizar a comunicação entre o Nível 1 e o Nível 3, que acontece nos dois sentidos em tempos diferentes.

Para mostrar o funcionamento do MoT em tempo real será utilizada a MAC centralizada em que o gateway inicia a comunicação e o nó sensor responde.

Para definição do MoT é necessário identificar as funções que o Nível 1:

- Medir alguma grandeza – no exemplo a luminosidade.
- Atuar no Nó Sensor – no exemplo acionar ou não o LED amarelo.

A seguir as partes necessárias para essas duas funções.



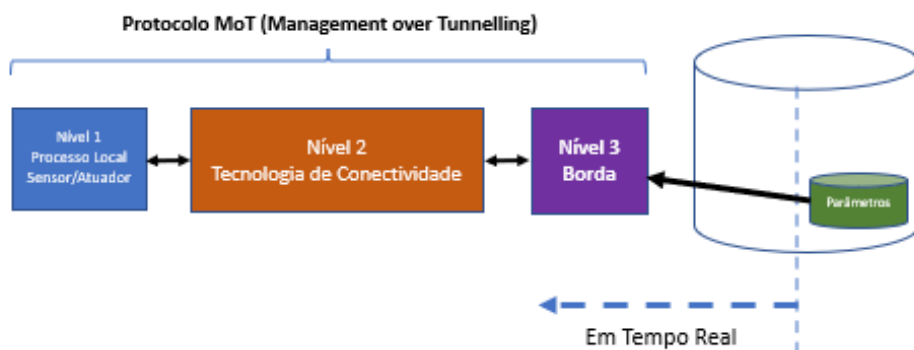
Nível 3 é um Python que tem duas funções: coletar e armazenar os dados brutos.

Nível 4 serão dois arquivos TXT:

- Arquivo de parâmetro que tem a ação a ser executada no Nó Sensor.
- Arquivo de dados brutos que terá a grandeza medida.

A seguir um detalhamento das operações que devem ser feitas pelo Nível 3.

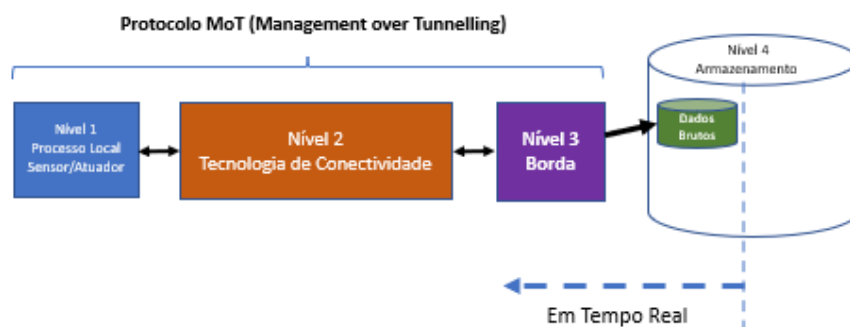
O Python do Nível 3 interage com o Nível 4 primeiramente acessando o arquivo de parâmetros para executar algum comando no Nível 1.



Nível 3 gera o pacote de DL com o comando do LED amarelo.

Nível 3 aguarda a resposta do Nível 1 com um pacote de UL com a informação da luminosidade.

Em seguida escreve no arquivo dados brutos do Nível 4 como dados brutos os pacotes de DL e UL criando um log de dados.



11 Pilha de Protocolo MoT

A definição de uma pilha de protocolos é fundamental para a comunicação entre os níveis 3 e 1.

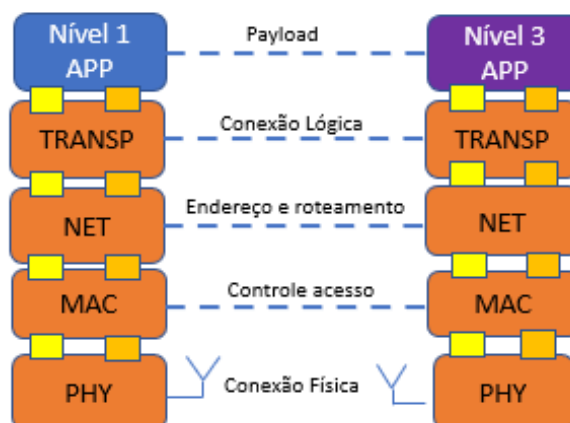
É necessário definir responsabilidades para criar rede de comunicação de dados, a exemplo do TCP/IP da Internet.

Entretanto, as informações de controle e gerência para uma RSSF possui diferentes necessidades, contempladas pelo MoT.

11.1 Pilha de Protocolo

Entender a lógica de como é utilizada a pilha de protocolo é fundamental para a construção de uma RSSF gerenciada.

Observar que as camadas estão ligadas uma a uma para gerenciar a RSSF.



A independência de cada par de camadas é fundamental para a evolução da pilha de protocolo.

Por exemplo, podem ser criadas estratégias para controle de taxa de sucesso de pacote na camada de transporte TRANSP, sem afetar as outras camadas.

Outro exemplo são estratégia de roteamento que estão na camada de rede NET.

11.2 Camada de aplicação – APP

Camada de aplicação na borda N3 que vai enviar o comando para ligar o desligar o LED amarelo. Essa informação está no arquivo comando_led_amarelo.txt que é lido pelo Python para acionar ou não o LED amarelo.

- DL – Envio de comando para o Nó Sensor para comando do LED
- UL – Recebe luminosidade

Camada de aplicação no Nó Sensor N1

- DL – Recebe o comando de controle do LED
- UL – Medida de luminosidade e envia para gateway

11.3 Camada de Transporte - TRANSP

Camada de transporte faz o controle de fluxo dos pacotes entre N3 e n1, por exemplo:

- Contagem de pacotes enviados.
- Avaliação de pacotes perdidos.
- Reenvio de pacotes perdidos.

11.4 Camada de Rede - NET

Camada de rede para endereçamento e roteamento:

- Cada elemento de uma rede de sensores sem fio deve ser identificado, por exemplo Sensor 1 e Base 0.
- Estratégia de roteamento para redes com múltiplos nós.

Exemplo de ações da camada de rede que é criar um híbrido LoRaWAN com LoRa.

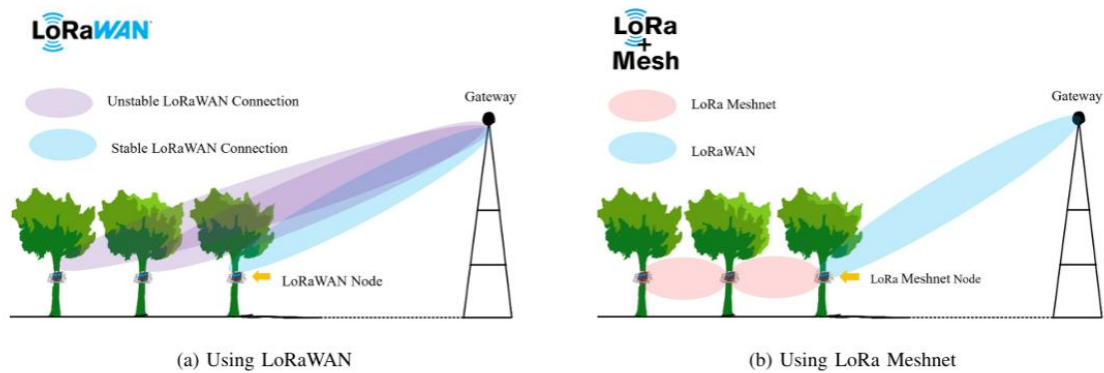


Fig. 1. The comparison of Fresnel zone between LoRaWAN and LoRa Meshnet.

11.5 Camada de acesso ao meio – MAC

São definidos os parâmetros ligados ao controle de acesso ao meio e energia. Exemplo:

- Tempo entre pacotes.
- Política de acesso ao meio.
- Gerência de energia do nó sensor
- Parametrização da interface rádio (por exemplo Spreading)

11.6 Camada física – PHY

Potência rádio:

- RSSI DL – medida no nó sensor
- RSSI UL – medida no gateway

Camada física com a definição da tecnologia de comunicação

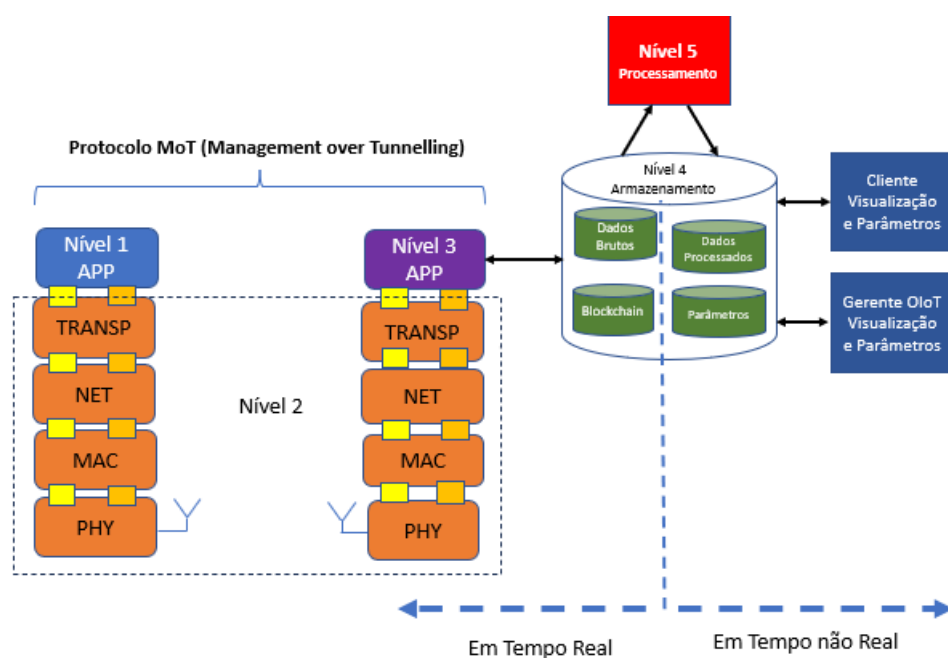
- LoRa
- LoRaWAN
- MQTT
- WiFi
- Bluetooth
- 4G
- USB

A camada física é a única que muda em função da tecnologia usada.

12 Framework com Pilha de Protocolo

Nesse item são mostradas todas as partes do Framework identificando a pilha do protocolo MoT para implementar o Nível 2.

A próxima figura mostra a pilha de protocolo MoT e todas as parte do Framework TpM.



Importante entender que a utilização de uma pilha de protocolo pelo Framework da TpM tem por objetivo criar todas as estratégias de gerência totalmente independentes do tipo de interface rádio.

12.1 Camada de Aplicação e Níveis do Framework

A figura mostra que para o Framework são identificados os níveis 1 e 2, que são as camadas de aplicação da pilha de protocolo.



Pode-se dizer que nesse ponto é o encontro entre o Framework e a pilha de protocolo.

Um ponto a destacar é que a camada de aplicação do nó sensor tem as funções:

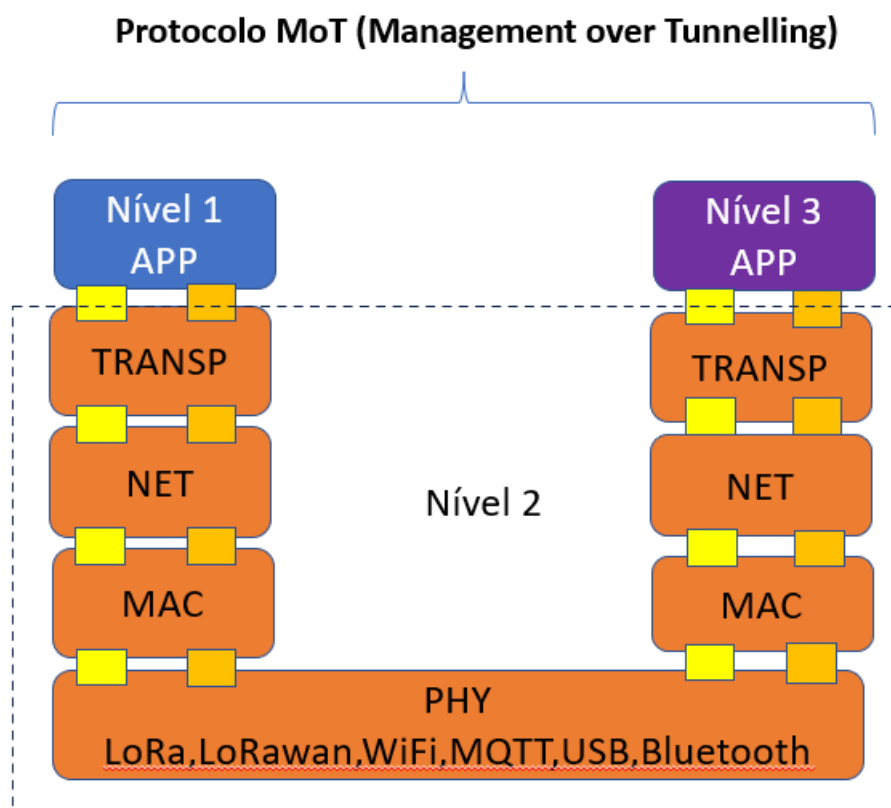
- Capturar grandezas.
- Executar comandos.

A camada de aplicação do Nível 3 tem por funções:

- Coletar grandezas vindas dos nós sensores.
- Enviar comandos para serem executados no nó sensor.

12.2 Independência da camada física com MoT

Uma importante função da pilha de protocolo do MoT é deixar as camadas MAC, NET, TRANSP independentes da camada física.



Com isso independentemente do tipo de camada física não existem alterações estruturais nas camadas superiores.

Por exemplo, a camada física pode ser LoRa, WiFi, Bluetooth, LoRaWAN, MQTT, etc.

Independente do tipo de solução de camada física não altera as camadas superiores.

13 Implementação com Kit-LoRa

Entender como foi projetado o Kit-LoRa é necessário para entender a implementação do exemplo Framework.

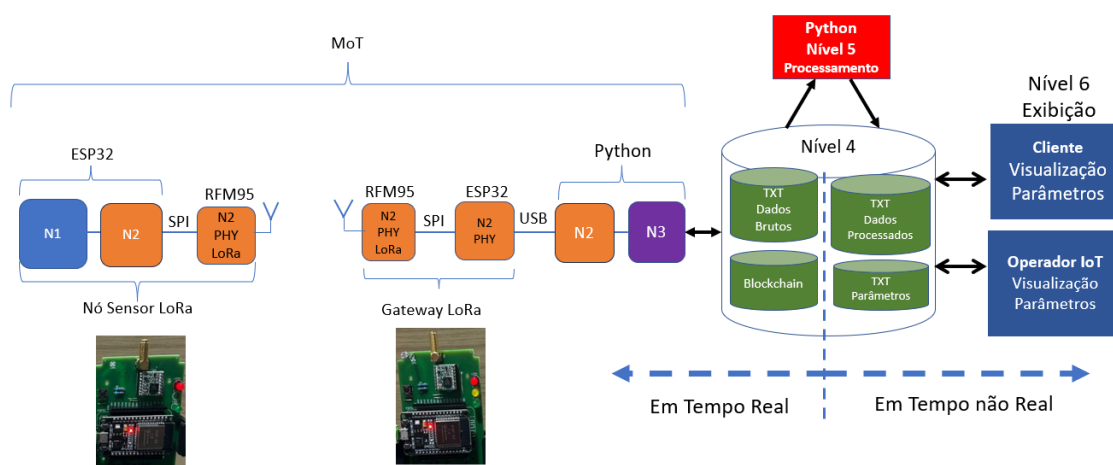
Kit-LoRa



13.1 Fluxo de dados N3 e N1

A identificação dos níveis do modelo de referência para IoT utilizado na TpM é necessário entender como é feito o mapeamento com o Framework.

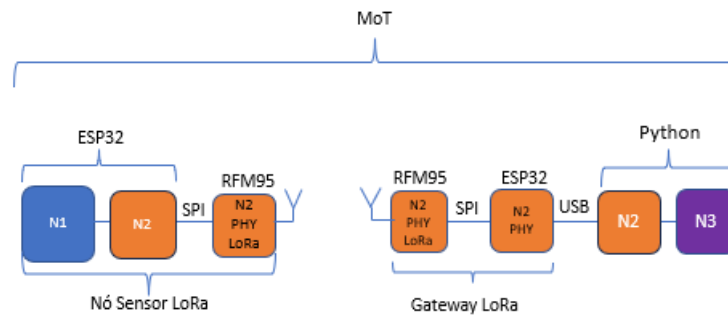
Para exemplificar a utilização da pilha de protocolo será utilizado o Kit-LoRa.



O “Motor” do framework é o Nível 3, que tem a dinâmica de operação em tempo real.

Entender como foi projetado o Kit-LoRa é necessário para entender como é implementada a pilha de protocolo.

Implementação.



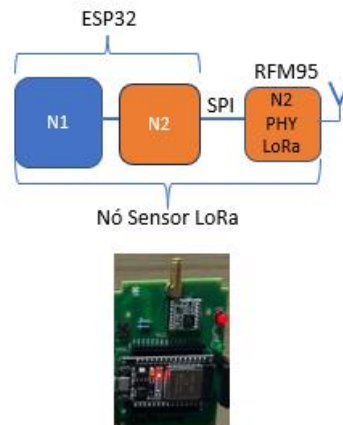
Definições:

- **Nó sensor** implementa no ESP32 a pilha de protocolo incluindo a parte do LoRa com o RFM95.
- **Gateway** somente parte da camada física com o ESP32 e o RFM95. Basicamente o gateway faz a ligação do sinal de rádio do LoRa com a USB do computador. Em especial o ESP32 é apenas uma ponte entre a USB e o SPI, nos dois sentidos.

13.2 Definição do Nó sensor – N1 e N2

O nó sensor é composto de duas partes:

- ESP32
- Módulo RFM95



O ESP32 tem duas funções:

- Nível 1 que faz os processos locais para acionamento do LED e medida da luminosidade. É esse nível que tem contato com as coisas.
- Nível 2 conectividade que tem as funções necessárias para permitir a comunicação através da interface LoRa.

O ESP32 está ligado ao módulo RFM95 através do protocolo SPI.

Esse módulo tem a modulação LoRa, além de outras modulações.

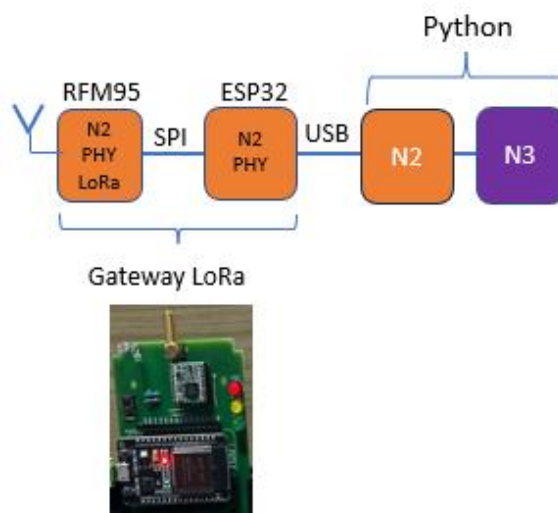
13.3 Definição do Gateway N2 Camada Física

Da mesma forma que o nó sensor o gateway é implementado utilizando:

- ESP32
- RFM95

Entretanto, não possui o LDR para medida de luminosidade e o LED amarelo para ser acionado.

O ESP32 tem a função de ligar a USB do computador com o módulo RFM95 via SPI, e vice versa.



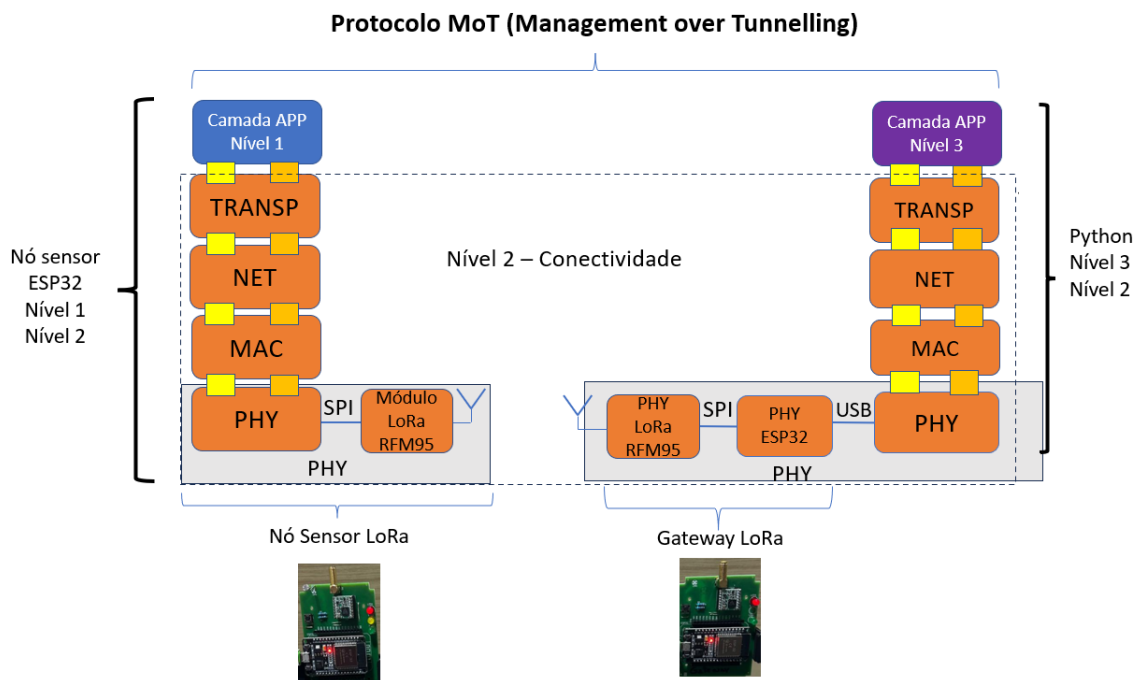
O Nível 2 no gateway tem, portanto, três partes:

- Computador com Python ligado via USB com o ESP32.
- ESP32 com código Arduino para ligação entre USB e SPI.
- Módulo RFM95 para comunicação com LoRa.

14 Pilha MoT com LoRa

A pilha MoT tem por objetivo permitir a gerência da conexão através da interface rádio LoRa.

A próxima figura mostra a implementação da pilha de protocolo MoT para o LoRa como interface rádio.



A figura permite compreender como é estruturada a camada física para a conectividade feita através do LoRa.

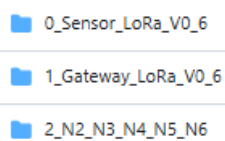
15 GitHub – Framework Basic

Para maior clareza de como é implementada uma solução IoT é feito um exemplo que está disponível no GitHub.

https://github.com/Professor-Omar-Branquinho/Framework_basic

Nesse item será explicado o **Framework Básico** utilizado como referência para todos os trabalhos do WissTek-IoT.

A organização dos subdiretórios.



A seguir como estão organizados os códigos.

16 Códigos Nó Sensor e Gateway

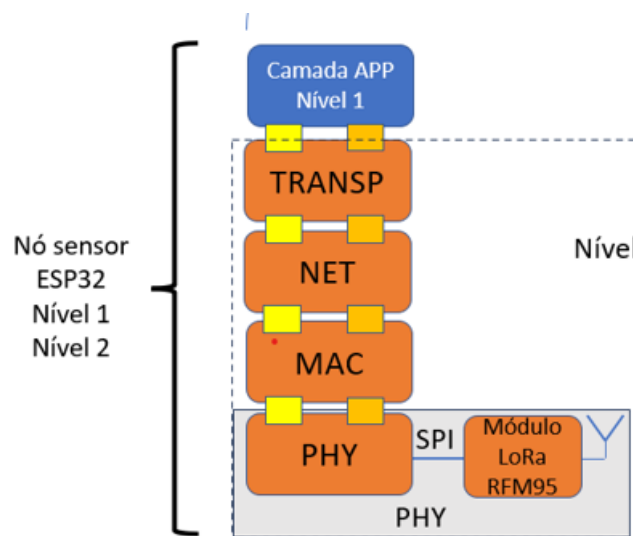
Para o Framework funcionar é necessário carregar os firmwares no nó sensor e no gateway.

Os códigos do nó sensor e gateway são criados no IDE do Arduino.

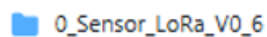


16.1 MoT no Nó Sensor

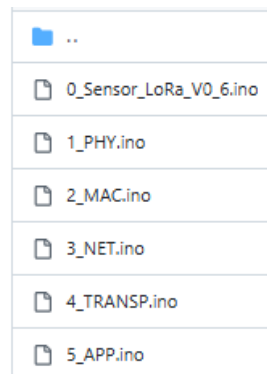
O ESP32 implementa os níveis 1 e 2 do modelo de referência para IoT, que estão no nó sensor e se comunica com o módulo RFM95 através da SPI.



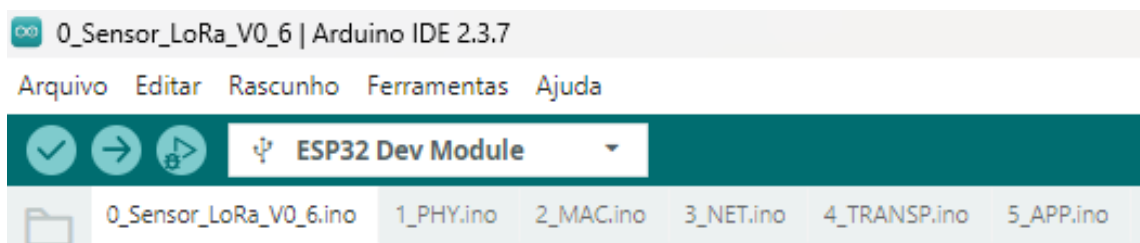
Subdiretório do nó sensor no GitHub:



O subdiretório tem os códigos do ESP32 que vai no nó sensor.



Código do nó sensor aberto no IDE do Arduino identificando as abas com o MoT.

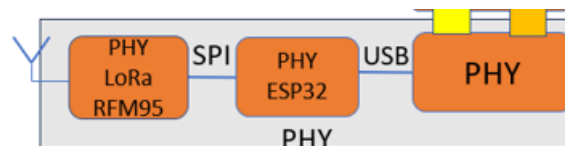


A utilização de abas tem por objetivo a separação das funções de cada camada da pilha MoT:

- Aba 1 – definições de variáveis, void setup(), void loop().
- Aba 2 – camada física com as informações sobre a interface rádio LoRa como: Spreading factor, bandwidth, Coding rate.
- Aba 3 – camada de controle de acesso ao meio.
- Aba 4 – camada de rede com identificação e estratégia de roteamento quando existir.
- Aba 5 – camada de transporte para controle de fluxo de pacotes do nó sensor.

16.2 Gateway

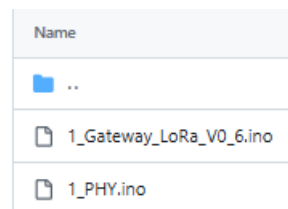
No ESP32 faz parte da camada física e tem por função a interface entre o módulo LoRa RFM95 através da SPI e o Python no computador via USB.



Subdiretório do gateway no GitHub:

[1_Gateway_LoRa_V0_6](#)

Somente é implementada a camada física, pois não tem nenhuma outra função implementada no gateway.



O gateway, diferentemente do nó sensor, tem somente a camada física para permitir a interface entre a comunicação LoRa e a USB do computador.

Código do gateway aberto no IDE do Arduino identificando as abas.



As informações de camada física para conexão com o RFM95 devem ser as mesmas utilizada no nó sensor.

O Gateway tem apenas parte da camada física, como já mencionado antes.

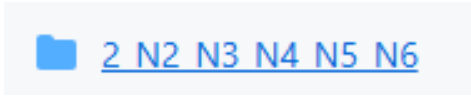
Mais a frente nesse material serão explicados os códigos do nó sensor e do gateway.

17 Pythons e Dados armazenados

Para apresentação da estrutura dos Pythons e dos dados armazenados será utilizado o exemplo que tem as funções:

- Coletar dados brutos e dados para visualização
- Processamento dos dados
- Exibição de gráficos e comandos.

No GitHub está na pasta:

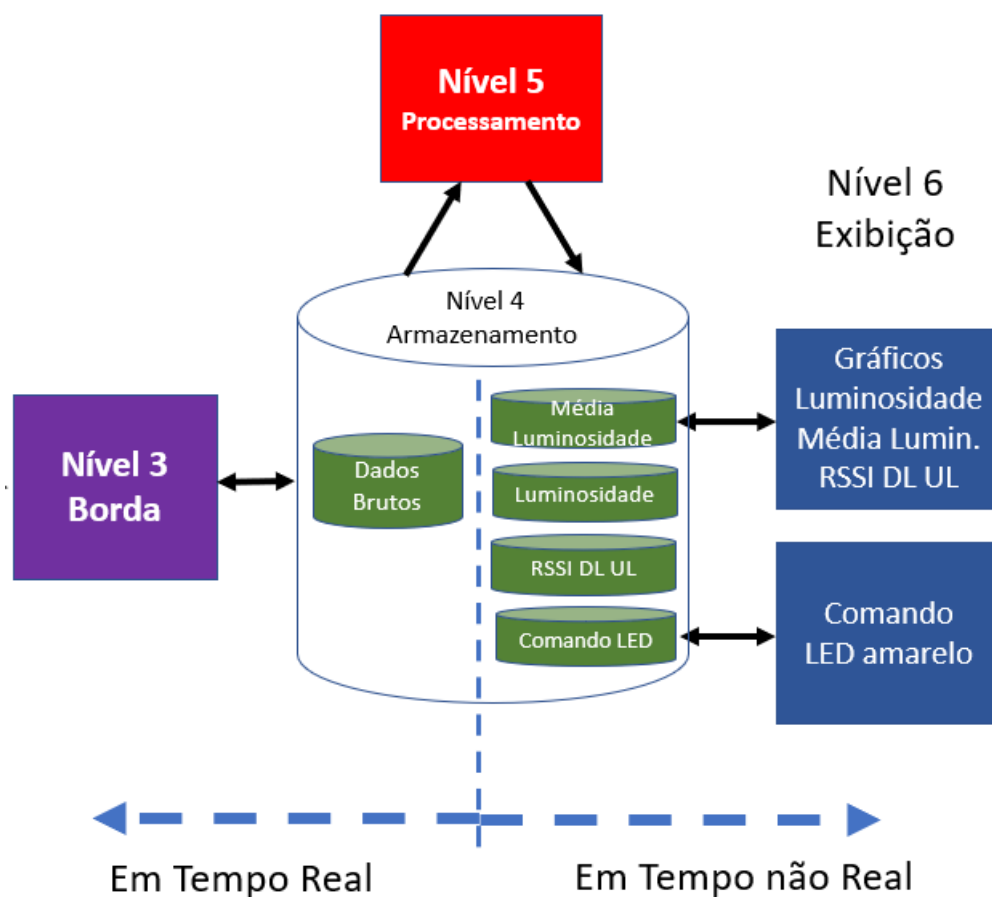


2 N2 N3 N4 N5 N6

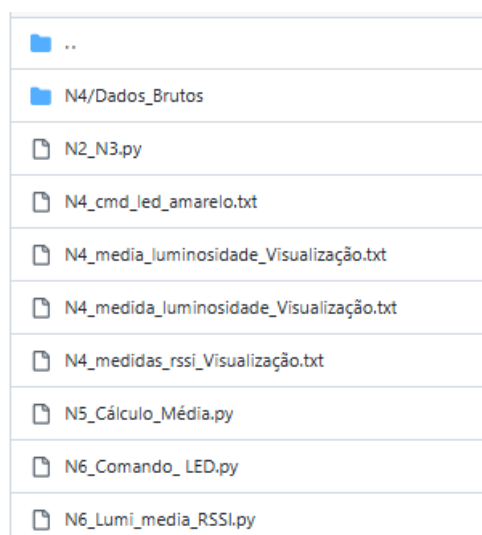
A organização dessa pasta segue o dos níveis do modelo de referência para IoT:

- N2 – Pilha MoT
- N3 – Coleta de dados e envio de comandos
- N4 – Estratégia de armazenamento
- N5 – Processamento de dados
- N6 – Exibição de gráficos e entrada de comandos

Considerando que o nó sensor e o gateway estejam com os códigos mostrados antes, a implementação do restante do modelo de referência para IoT com níveis 2, 3, 4, 5, 6 é o seguinte:



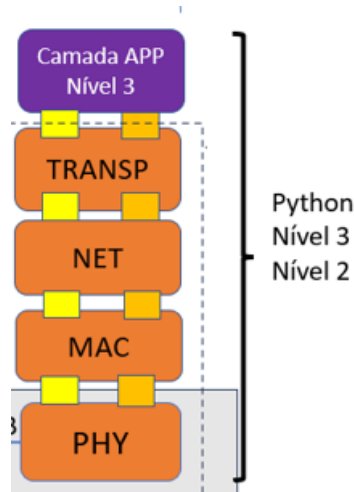
No subdiretório no GitHub estão, além dos Pythons de níveis 2_3, 5 e 6 estão também os dados armazenados.



A seguir cada arquivo é explicado.

17.1 N2_N3.py – MoT e Aplicação

O Python N2_N3 implementa os níveis 2 e 3 do modelo de referência para IoT.



Esse Python executa em tempo real a consultado do arquivo de comando do LED e envia para o nó sensor no pacote de DL.

Recebe o pacote de UL e armazena:

- Dados brutos em uma pasta separada
- Dados para visualização
 - Luminosidade em tempo real
 - RSSI em tempo real

17.2 N4 - Armazenamento

O Nível 4 tem a função lógica de identificar todas as informações que precisam ser armazenadas.

Os dados para serem armazenados são gerados pelos outros níveis:

- Nível 3
 - dados brutos armazenados em diretório específico com as informações dos pacotes de DL e UL
 - dados para visualização em tempo real
- Nível 5 – dados processados devem ser armazenados

- N6 – entrada de comandos deve ser armazenado

17.2.1 Comando do LED Amarelo

O N6 tem um Python que gera o comando para o LED amarelo e armazena esse comando no arquivo:

 N4_cmd_led_amarelo

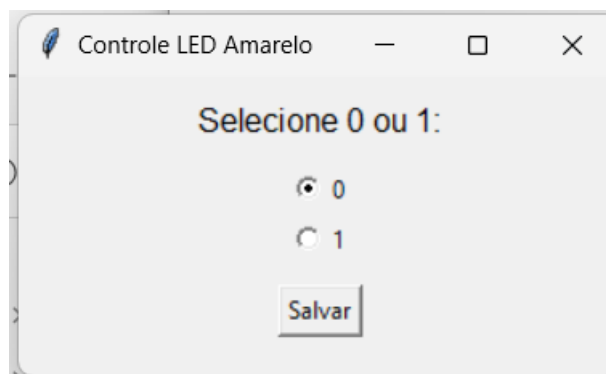
Esse é um arquivo TXT que possui apenas um valor:

- 0 – LED amarelo apagado
- 1 – LED amarelo aceso

Esse arquivo é criado e atualizado pelo Python:

 N6_Comando_LED.py

A executar esse Python será dada a opção de alterar o status do LED amarelo.



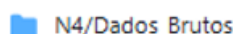
Esse é um Python bastante simples gerado por IA.

Basicamente escreve no arquivo N4_cmd_led_amarelo o valor 0 ou 1.

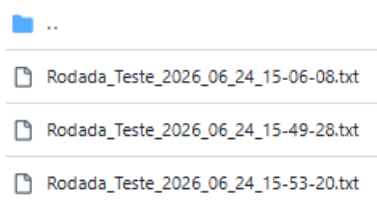
17.2.2 Dados Brutos

Os dados brutos são armazenados pelo N3 que envia o comando para ligar ou desligar o LED amarelo e coleta a luminosidade e a RSSI.

Existe um subdiretório específico para armazenamento dos dados brutos.



Nesse subdiretório são armazenados os dados obtidos em cada rodada de medidas com data e hora para serem utilizados no futuro para análise.



Os logs tem a seguinte estrutura em cada linha:

Timestamp	Contador Pacotes DL	Pacote DL – 20 Bytes	Pacote UL – 20 Bytes
-----------	---------------------	----------------------	----------------------

Exemplo de log:

Wed	Jun	24	15:06:14	2026,	1,0	,0	,0	,0	,0	,2	,0	,0	,0	,1	,0	,0	,0	,1	,0	,0	,0	,0	,0	,84	92	82	95	,0	,0	,0	,0	,0	,1	,0	,0	85	,0	,0	,0	31	,0	93
Wed	Jun	24	15:06:16	2026,	2,0	,0	,0	,0	,0	,2	,0	,0	,0	,1	,0	,0	,0	,2	,0	,0	,0	,0	,0	84	97	84	95	,0	,0	,0	,0	,0	1	,0	,0	86	,0	,0	,0	31	,0	93
Wed	Jun	24	15:06:18	2026,	3,0	,0	,0	,0	,0	,2	,0	,0	,0	1	,0	,0	,0	3	,0	,0	,0	,0	,0	84	95	84	95	,0	,0	,0	,0	,0	1	,0	,0	87	,0	,0	,0	31	,0	93
Wed	Jun	24	15:06:20	2026,	4,0	,0	,0	,0	,0	,2	,0	,0	,0	1	,0	,0	,0	4	,0	,0	,0	,0	,0	84	95	82	95	,0	,0	,0	,0	,0	1	,0	,0	88	,0	,0	,0	31	,0	93
Wed	Jun	24	15:06:22	2026,	5,0	,0	,0	,0	,0	2	,0	,0	,0	1	,0	,0	,0	5	,0	,0	,0	,0	,0	84	92	84	95	,0	,0	,0	,0	,0	1	,0	,0	89	,0	,0	,0	31	,0	95
Wed	Jun	24	15:06:24	2026,	6,0	,0	,0	,0	,0	2	,0	,0	,0	1	,0	,0	,0	6	,0	,0	,0	,0	,0	84	92	84	92	,0	,0	,0	,0	,0	1	,0	,0	90	,0	,0	,0	31	,0	85
Wed	Jun	24	15:06:26	2026,	7,0	,0	,0	,0	,0	2	,0	,0	,0	1	,0	,0	,0	7	,0	,0	,0	,0	,0	84	95	84	95	,0	,0	,0	,0	,0	1	,0	,0	91	,0	,0	,0	31	,0	91
Wed	Jun	24	15:06:28	2026,	8,0	,0	,0	,0	,0	2	,0	,0	,0	1	,0	,0	,0	8	,0	,0	,0	,0	,0	84	95	82	97	,0	,0	,0	,0	,0	1	,0	,0	92	,0	,0	,0	31	,0	85
Wed	Jun	24	15:06:30	2026,	9,0	,0	,0	,0	,0	2	,0	,0	,0	1	,0	,0	,0	9	,0	,0	,0	,0	,0	86	92	86	97	,0	,0	,0	,0	,0	1	,0	,0	93	,0	,0	,0	31	,0	84
Wed	Jun	24	15:06:32	2026,	10,0	,0	,0	,0	,0	2	,0	,0	,0	1	,0	,0	,0	10	,0	,0	,0	,0	,0	86	95	84	95	,0	,0	,0	,0	,0	1	,0	,0	94	,0	,0	,0	31	,0	85
Wed	Jun	24	15:06:34	2026,	11,0	,0	,0	,0	,0	2	,0	,0	,0	1	,0	,0	,0	11	,0	,0	,0	,0	,0	86	92	84	97	,0	,0	,0	,0	,0	1	,0	,0	95	,0	,0	,0	31	,0	82
Wed	Jun	24	15:06:36	2026,	12,0	,0	,0	,0	,0	2	,0	,0	,0	1	,0	,0	,0	12	,0	,0	,0	,0	,0	86	92	86	92	,0	,0	,0	,0	,0	1	,0	,0	96	,0	,0	,0	31	,0	90
Wed	Jun	24	15:06:38	2026,	13,0	,0	,0	,0	,0	2	,0	,0	,0	1	,0	,0	,0	13	,0	,0	,0	,0	,0	86	92	84	92	,0	,0	,0	,0	,0	1	,0	,0	97	,0	,0	,0	31	,0	86
Wed	Jun	24	15:06:40	2026,	14,0	,0	,0	,0	,0	2	,0	,0	,0	1	,0	,0	,0	14	,0	,0	,0	,0	,0	86	95	84	92	,0	,0	,0	,0	,0	1	,0	,0	98	,0	,0	,0	31	,0	85
Wed	Jun	24	15:06:42	2026,	15,0	,0	,0	,0	,0	2	,0	,0	,0	1	,0	,0	,0	15	,0																							

Os arquivos de log tem por função ter todas as informações trocadas com o nó sensor pelo gateway, que podem ser resgatadas no futuro para processamento.

17.2.3 Dados de Visualização

O exemplo utiliza para demonstração os gráficos da luminosidade, média da luminosidade e RSSI, que são criados da seguinte maneira:

O N3 armazena os dados:

- Luminosidade em tempo real
- RSSI em tempo real

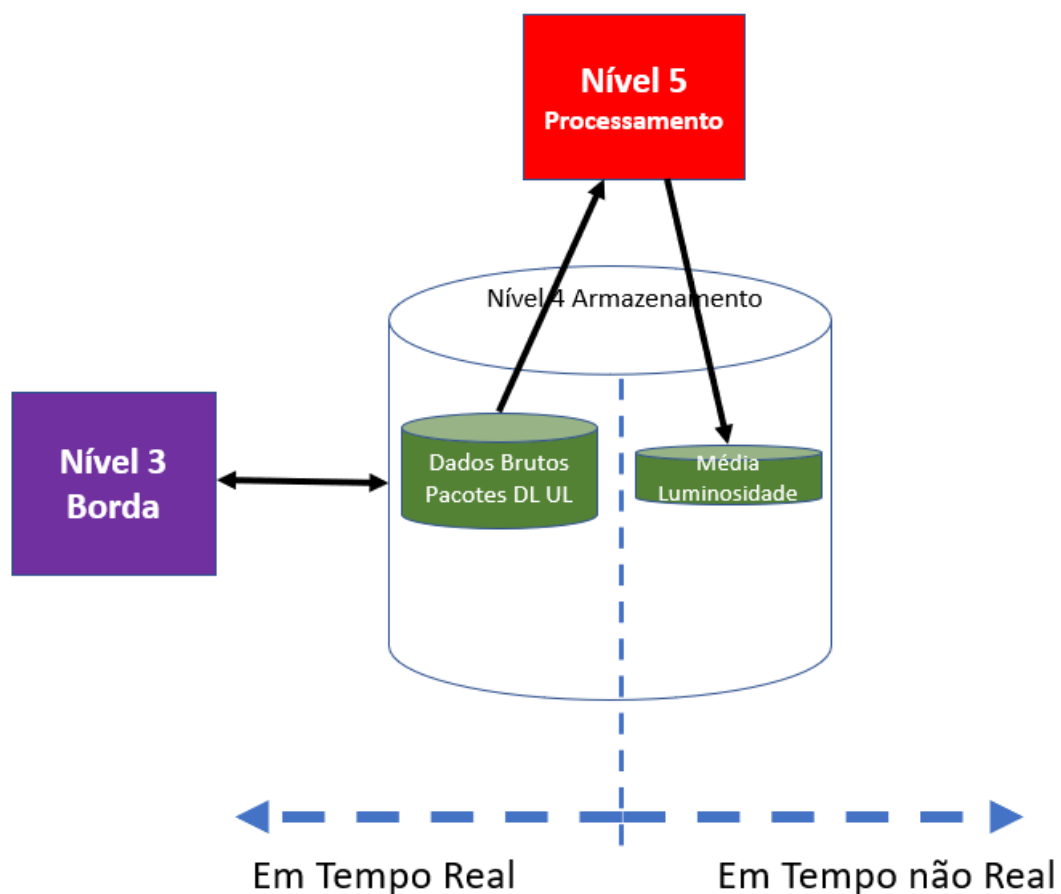
O N5 calcula a média.

Os arquivos de visualização serão lidos para exibição em tempo real da luminosidade, sua média e RSSI.

 N4_media_luminosidade_Visualização
 N4_medida_luminosidade_Visualização
 N4_medidas_rssi_Visualização

17.3 N5 - Processamento

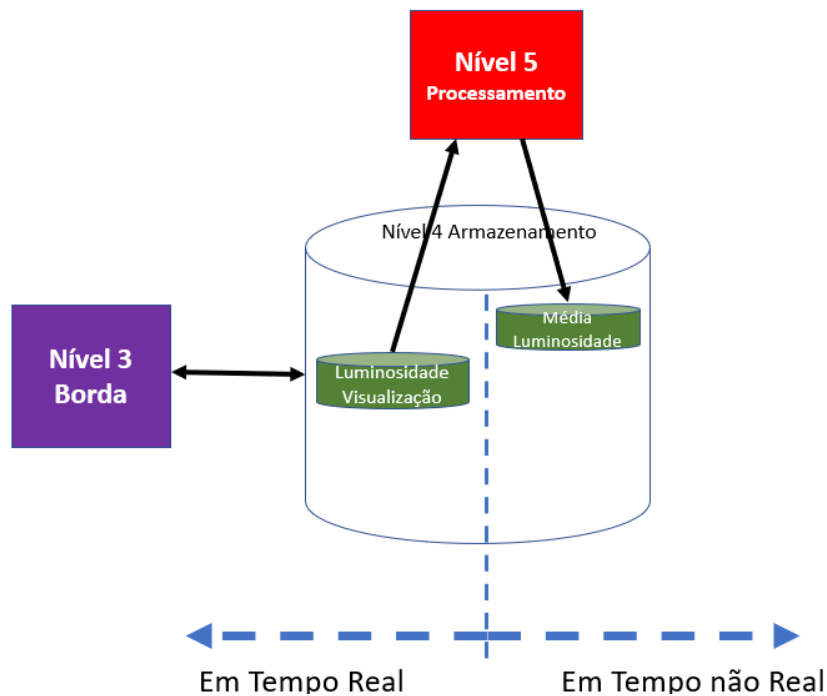
O N5 no exemplo tem o objetivo de calcular a média da luminosidade.



A média da luminosidade pode ser obtida lendo os dados brutos em tempo real.

Outra alternativa é criar arquivos de visualização para a luminosidade, média da luminosidade e RSSI de DL e UL.

Com essa opção o diagrama fica como a seguir.



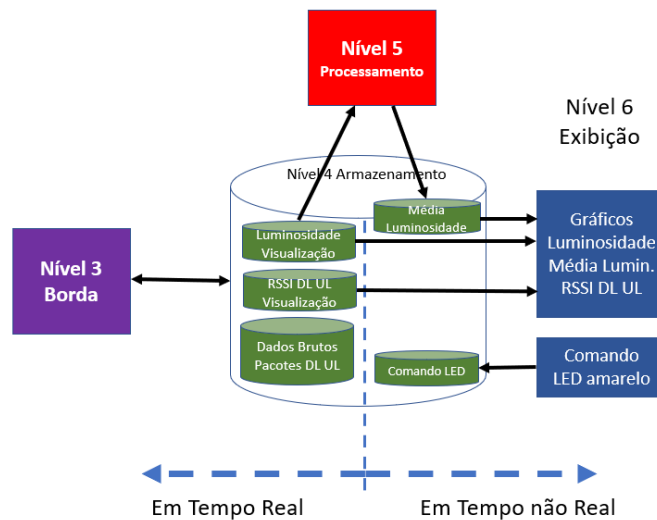
Nessa configuração Python N5 lê o arquivo:

 N4_medida_luminosidade_Visualização.txt

Calcula a média da luminosidade.

Esse é um artifício para não ser necessário ler os dados brutos, que ficam para análises posteriores.

17.4 N6 – Exibição



18 Exemplo Passo a Passo do Framework Básico

Nesse item são apresentados todos os passos para entender o exemplo do Framework Básico.

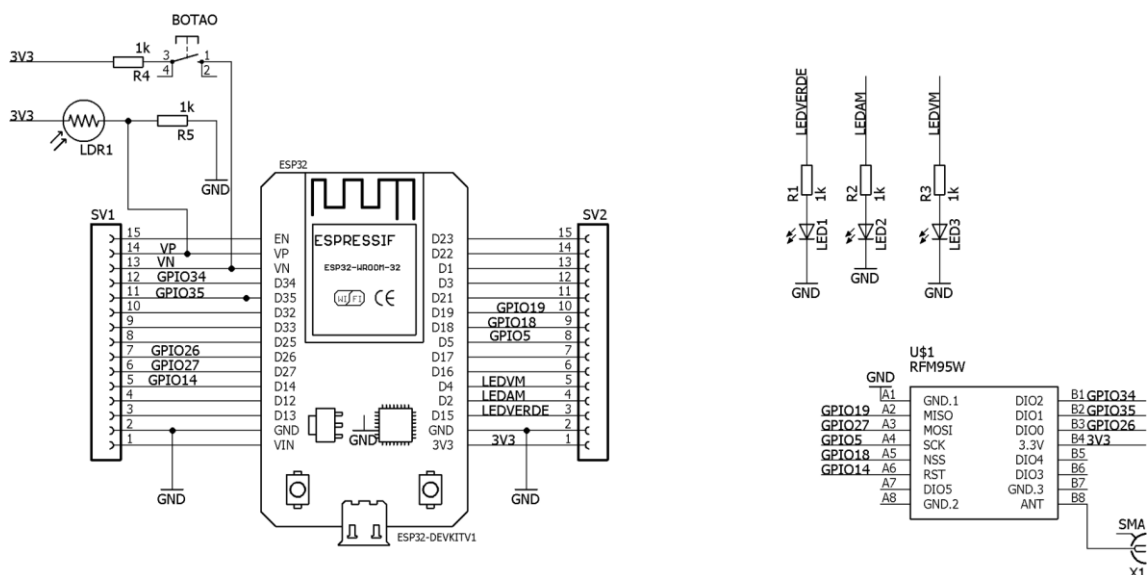
Fases da implementação de uma S-IoT

- Finalidade
- Projeto do nó sensor
 - Esquema elétrico
 - Layout
 - Implementação
- Projeto dos pacotes DL e UL
- Firmware nó sensor N1 e N2
- Firmware gateway parte camada física borda
- Python N2 e N3
- Estratégia de armazenamento N4
- Processamento dos dados N5
- Exibição N6

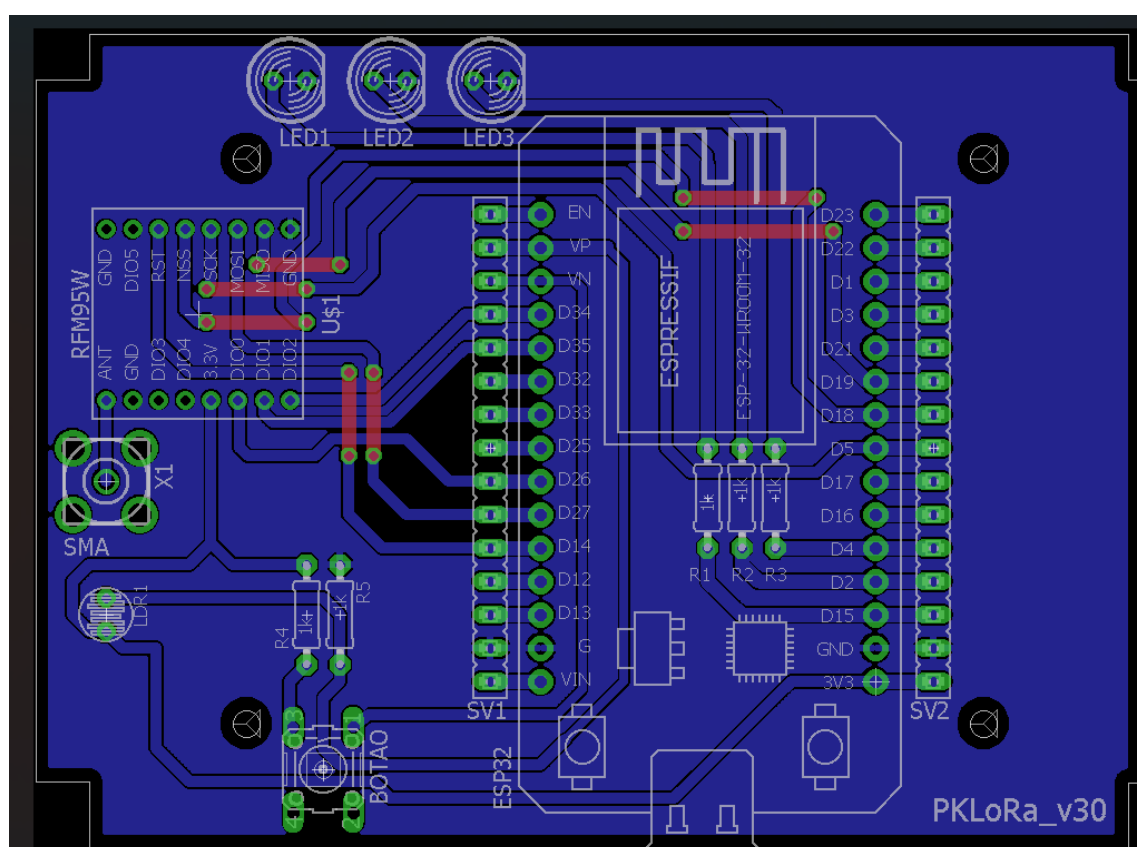
○

18.1 Projeto do Nó sensor

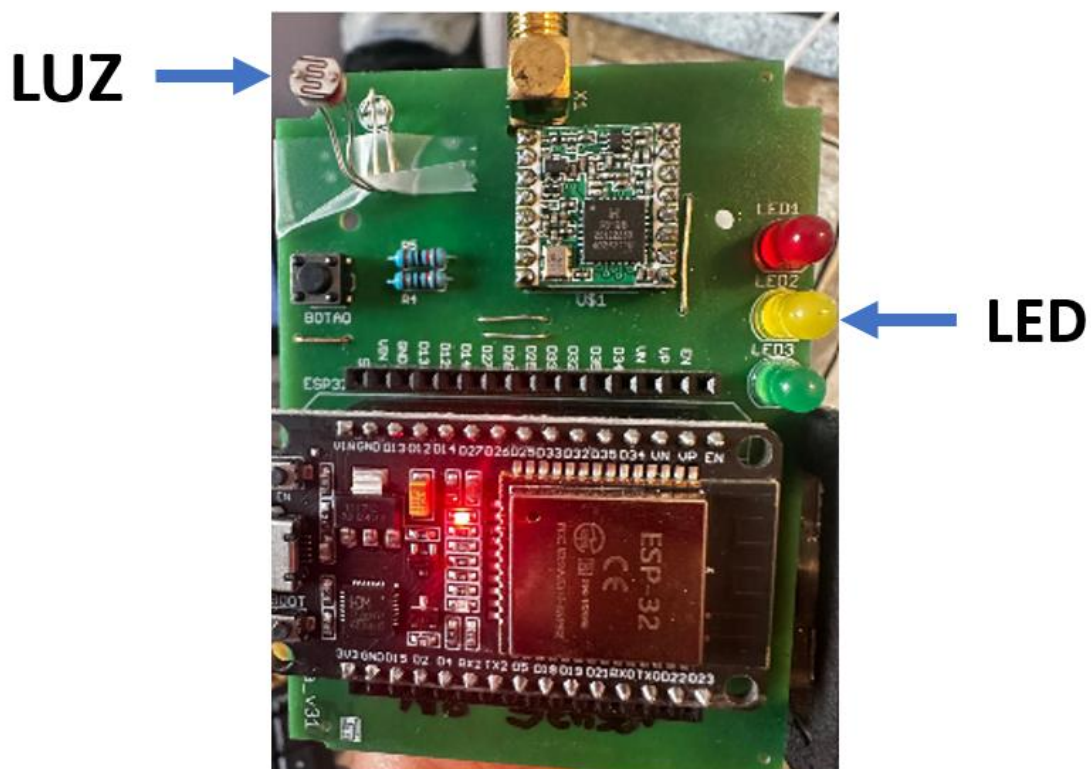
É necessário projetar um nó sensor que permita medir luz e comandar um LED.



Layout.



Implementação do nó sensor.



18.2 Pacote MoT DL e UL

Por questão didática o pacote é definido com o mesmo tamanho de 20 bytes de DL e UL.

Byte	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
Função	PHY				MAC				NET				TRANSP				APP			

Observar que o pacote pode ser bem menor que 20 bytes, mas por questões didáticas serão utilizados 20 bytes:

- Camada PHY – 4 bytes
- Camada MAC – 4 bytes
- Camada NET – 4 bytes
- Camada TRANSP – 4 bytes
- Camada APP – 4 bytes



O tamanho do pacote em uma Solução IoT vai depender das necessidades de medidas, controle e gerência.

Observar que o número do byte vai de 0 a 19, portanto 20 bytes.

As informações que vão nos bytes de DL e UL são diferentes em função das necessidades para cada direção de comunicação.

Por questões didáticas foi mantido o mesmo tamanho dos pacotes de DL e UL.

Nesse item é projetado o pacote para o Framework Básico que tem as seguintes funções

- Acionar um LED amarelo com um comando com pacote DL
- Medir a luminosidade proporcional com pacote UL

18.3 Pacote de DL

O pacote de DL leva informações de controle na camada de aplicação, que no caso é o comando do LED amarelo do Nó Sensor.



A seguir a definição das informações de cada byte do pacote de DL que está sendo utilizado pelo Kit-LoRa.

Pacote de Down Link - DL		
PHY	0	TBD
	1	TBD
	2	TBD
	3	TBD
MAC	4	Tempo entre pacotes
	5	TBD
	6	TBD
	7	TBD
NET	8	Identificação do sensor de destino
	9	Identificação do gateway de origem
	10	TBD
	11	TBD
TRANSP	12	Número de pacotes DL
	13	TBD
	14	TBD
	15	TBD
Dados DL	16	Comando LED amarelo
	17	TBD
	18	TBD
	19	TBD

A sigla TBD é para ser determinado no futuro.

18.4 Pacote de UL

O pacote de UL é criado pelo Nível 1 e está dividido também em Header e Payload.



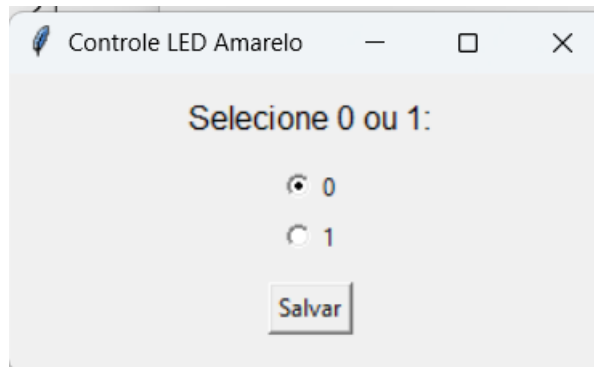
A seguir a definição das informações de cada byte do pacote de UL que está sendo utilizado pelo Kit-LoRa.

Pacote de Up Link - UL		
PHY	0	RSSI DL
	1	SNR DL
	2	RSSI UL
	3	SNR UL
MAC	4	Energia da bateria do nó sensor (Não implementado)
	5	TBD
	6	TBD
	7	TBD
NET	8	Identificação do gateway destino do pacote
	9	Identificação do nó sensor de origem
	10	TBD
	11	TBD
TRANSP	12	Número de pacote UL
	13	Perda de pacote DL
	14	TBD
	15	TBD
Dados UL	16	Status do LED
	17	Código do tipo de hardware PK-LoRa
	18	Inteiro luminosidade
	19	Resto luminosidade

18.5 Acionamento do LED Amarelo

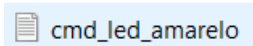
O primeiro Python a ser executado é o N6_LED.

O Python que liga ou desliga o LED amarelo é bastante simples e tem a seguinte tela.



Esse Python cria um arquivo de comando do LED e dá opção para ligar com valor 1 e desligar com valor 0.

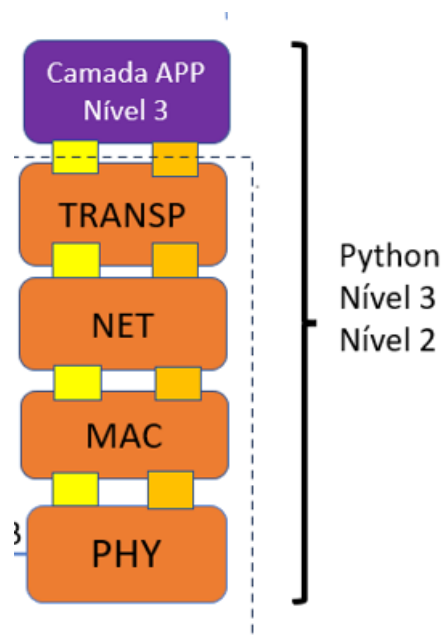
O arquivo criado no Nível 4 de armazenamento do status do LED:



Esse é um arquivo de uma linha e uma coluna com o valor 0 ou 1, escrito quando clica em gravar.

18.6 Python N2 e N3

Em seguida é executado o Python N2_N3.



O Python que é o “Motor” do Framework tem por função gerar o pacote de DL e receber o pacote de UL.

Importante, o MoTTest deve ser fechado para funcionar o Python N2_N3.

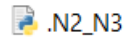
Ao executar esse Python é solicitada a COM do gateway e o número de medidas a serem feitas.

```
Digite o número da serial = 8
Entre com o número de medidas = 1000
```

Sequência de ações:

18.7 Leitura do Comando do LED Amarelo

O arquivo de comando do LED amarelo é lido no código do Python:



O trecho do código que faz essa leitura do arquivo do comando do LED amarelo.

```
# ===== Camada de aplicação PACOTE DL
# Lê o arquivo cmd_led_amarelo.txt
with open("cmd_led_amarelo.txt", "r") as f:
    linha = f.readline()
    # Remove espaços e ENTER
    linha = linha.strip()
    # Se o valor for 0 ou 1
    if linha == "0":
        Comando_LED_amarelo = 0
    elif linha == "1":
        Comando_LED_amarelo = 1
    else:
        # Qualquer outro conteúdo assume 0
        Comando_LED_amarelo = 0
except:
    # Se houver qualquer erro assume 0
    Comando_LED_amarelo = 0
# Coloca o comando no byte 16 do DL
Pacote_DL[16] = Comando_LED_amarelo
```

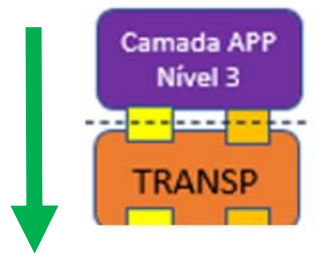
Será utilizado a seguir a sequência de down link completa:

- Python – Nível 3
- Firmware ESP2 gateway – Nível 2
- Firmware ESP32 nó sensor – Nível 2 e Nível 1

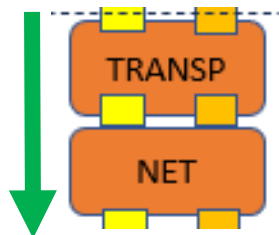
Camada APP DL – na camada de aplicação o pacote de DL envia o comando para ligar ou desligar o LED amarelo através do Byte 16.

Fica claro o desperdício na utilização de um byte para ligar e desligar um LED.

Nos desenvolvimentos é necessário utilizar o byte com mais eficiência levando mais informações.

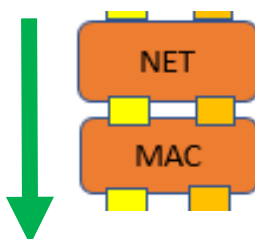


Camada TRANS DL – informa no Byte 12 o número de pacotes já enviados pelo Nível 3. Nesse caso foi usado apenas um byte por simplicidade e no nó sensor é necessário tratar esse contador quando chega em 255.

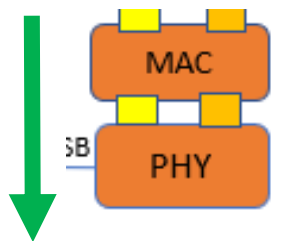


Camada NET DL – informa a identificação dos elementos de rede:

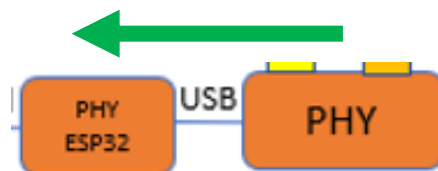
- Byte 8 – identificação do nó sensor que deve receber o pacote
- Byte 9 – identificação da base que originou o pacote de DL.



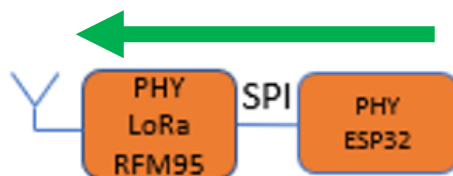
Camada MAC DL – informa no Byte 4 o intervalo entre os pacotes que serão transmitidos pelo Nível 3. Essa é uma informação importante para que o Nó sensor, eventualmente, entre em sleepmode, criando uma estratégia para economia de energia.



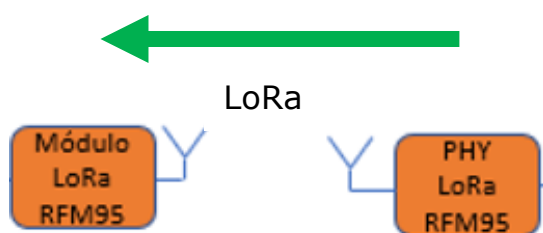
Camada PHY DL – Na evolução do protocolo MoT na camada PHY serão colocados os parâmetros do LoRa. O pacote é disponibilizado pela USB para ser lido pelo ESP32.



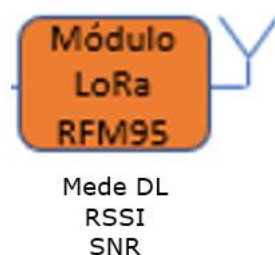
O ESP32 lê pela USB o pacote de 20 bytes e envia para o módulo RFM95 via SPI.



Após a transmissão pela interface LoRa é recebido pelo nó sensor.



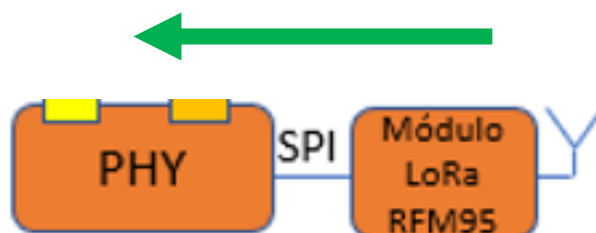
O RFM95 mede a RSSI DL e a SNR.



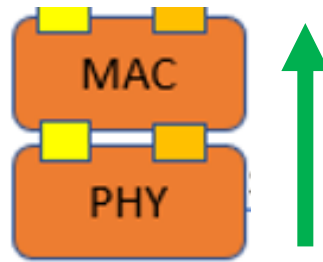
RFM95 disponibiliza dos bytes do pacote DL, RSSI e SNR para serem lidos pelo ESP32 pela SPI.

A leitura será feita pelo firmware do ESP32, como será explicado no código.

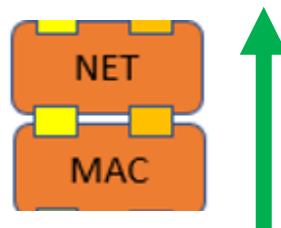
Camada PHY DL – leitura do pacote de DL, RSSI e SNR.



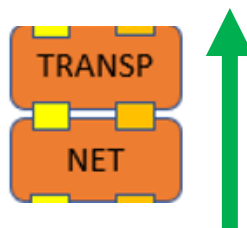
Camada MAC DL – leitura do tempo entre pacote no Byte 4 para sincronização.



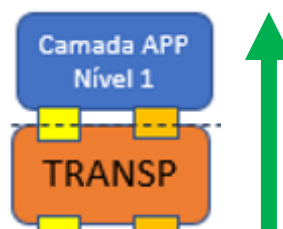
Camada NET DL – verifica a identificação no Byte 8 para checar se o pacote foi endereçado para esse nó sensor e enviado pelo gateway no Byte 9.



Camada TRANSP DL – camada de transporte recebe no Byte 12 o número de pacotes de DL e calcula se perdeu algum pacote de DL com parando com o byte anterior de DL.

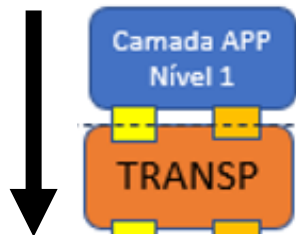


Camada APP DL – a camada de aplicação executa o comando de apagar ou ascender o LED amarelo.



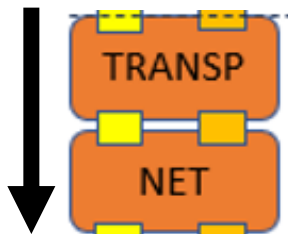
Interessante observar que a camada de aplicação da pilha de protocolo se confunde com o Nível 1 do modelo de referência.

Camada APP UL - Coloca o status do LED no Byte 16 de UL. Identifica através de um código qual o hardware e coloca no Byte 17, no caso a PK-LoRa. Faz medida da luminosidade e aloca inteiro no Byte 18 e resto no Byte 19.



Camada de TRANSP UL – a camada de transporte faz o controle de fluxo contabilizando o número de pacotes enviados e recebidos e contabilização de perdas de pacote.

- Byte 12 – número de pacote enviados de UL.
- Byte 13 – número de pacotes perdidos de DL.

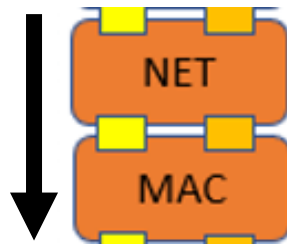


Camada NET UL – informa a identificação dos elementos de rede:

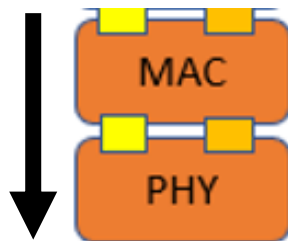
Byte 8 – identificação do gateway que deve receber o pacote de UL.

Byte 9 – identificação DO que originou o pacote de DL.

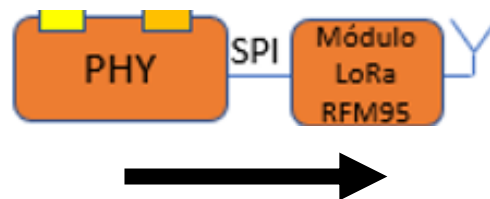
Em desenvolvimentos com múltiplos sensores é nessa camada que será criada a estratégia de roteamento.



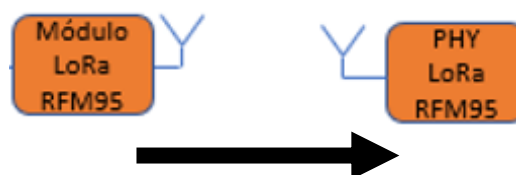
Camada MAC UL – informa no Byte 4 a energia das baterias do nó sensor. Na versão didática apresentada não está sendo gerada essa informação.



ESP32 envia pelo SPI o pacote para o RFM95.



Pacote é transmitido pela interface LoRa

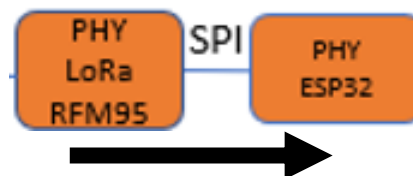


Camada PHY UL – Pacote é recebido pelo módulo RFM95 da borda. RFM95 mede RSSI UL e RSSI UL. Disponibiliza o pacote de UL para ser lido pelo ESP32.



Mede UL
RSSI
SNR

Leitura pelo ESP32 via SPI do RFM95 do pacote UL com 20 bytes, RSSI UL e SNR UL.

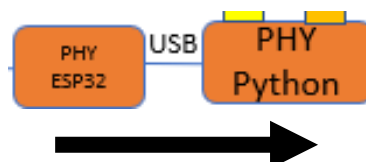


Escreve nos bytes

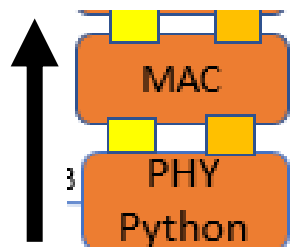
- Byte 2 – RSSI de UL medida no nó sensor
- Byte 3 – SNR de UL medida no nó sensor

Lembrando que nos bytes 0 e 1 estão RSSI DL e SNR DL, respectivamente.

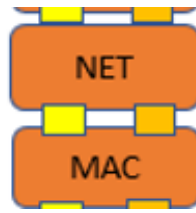
ESP32 disponibiliza o pacote com 20 bytes para ser lido pela camada PHY do Python.



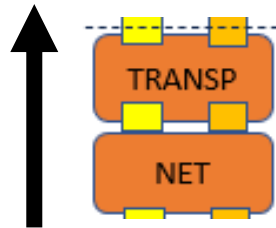
Camada MAC UL – recebe o nível de bateria no Byte 2.



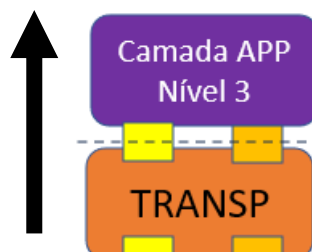
Camada NET UL – verifica a identificação do sensor que enviou o pacote no Byte 9 e verifica se o pacote é para essa borda no Byte 8.



Camada TRANSP UL – checa a contagem de pacotes de UL no Byte 12.
Verifica o número de pacotes perdidos no DL Byte 13.



Camada APP UL – a camada de aplicação no UL tem por função organizar e armazenar os dados brutos coletados em tempo real no Nível 4.



19 Organização Dados Brutos

Esses dados são divididos em dois tipos:

- Dados para o cliente.
- Dados para gerência da RSSF.

